

Using business process models to better understand the dependencies among user stories



Marina Trkman^{a,*}, Jan Mendling^b, Marjan Krisper^a

^a University of Ljubljana, Faculty of Computer and Information Science, Večna pot 113 SI-1001 Ljubljana, Slovenia

^b WU (Vienna University of Economics and Business), Department of Information Systems and Operations, Institute for Information Business, Welthandelsplatz 1, 1020 Vienna, Austria

ARTICLE INFO

Article history:

Received 1 June 2015

Revised 16 October 2015

Accepted 19 October 2015

Available online 27 November 2015

Keywords:

User story

Execution order dependency

Integration dependency

Context

Experiment

ABSTRACT

Context: Agile software development projects often manage user requirements with models that are called user stories. Every good user story has to be independent, negotiable, valuable, estimable, small, and testable. A proper understanding of a user story also requires an understanding of its dependencies. The lack of explicit representation of such dependencies presumably leads to missing information regarding the context of a user story.

Objective: We propose a method that facilitates better understanding of execution order and integration dependencies of user stories by making use of business process models. The method associates user stories with the corresponding business process model activity element.

Method: We adopted a situational method engineering approach to define our proposed method. In order to provide understanding of proposed method's constructs we used ontological concepts. Our method associates a user story to an activity element. In this way, the business process model can be used to infer information about the execution order and integration dependencies of the user story. We defined three levels of association granularity: a user story can be more abstract, approximately equal to, or more detailed than its associated business process model activity element. In our experiment we evaluate each of these three levels.

Results: Our experiment uses a between-subject design. We applied comprehension, problem-solving and recall tasks to evaluate the hypotheses. The statistical results provide support for all of the hypotheses. Accordingly, there appears to be significantly greater understanding of the execution order and integration dependencies of user stories when associated business process models are available.

Conclusions: We addressed a problem which arises from managing user stories in software development projects and focuses on the missing context of a user story. Our method contributes to the discipline of conceptual modeling in agile development. Our experiment provides empirical insight into requirement dependencies.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

User stories are the most commonly used requirement model in agile development projects [1–3]. Every good user story has to be independent, negotiable, valuable, estimable, small, and testable (INVEST) [4–7]. How to specify a good user story is well covered by the professional literature [5,6,8]. However, a single user story does not convey the entire business, but only a part of it [10]. This means that, while a user story is independent from the perspective of a developer, it can still depend on other user stories from a business perspec-

tive [10]. In this context, insights into requirement dependencies are crucial for project success [9–15] and more specifically for requirements interaction management [16]. Being unaware of requirement dependencies can lead to missing information regarding the context of a project, namely, its domain [5,17]. Martakis and Daneva [10] have emphasized the importance of integration and execution order dependencies. *Integration dependencies* present information about how certain user stories require other user stories to be previously developed. While *execution order dependencies* present information about how the completion of a user story directly impacts another. Strode [34] refers to these two types of requirements as technical dependencies and activity dependencies.

In practice, it is very important to understand if there are any dependencies among user stories and why they exist. Tracing execution

* Corresponding author.

E-mail addresses: marina.trkman@gmail.com (M. Trkman), jan.mendling@wu.ac.at (J. Mendling), marjan.krisper@fri.uni-lj.si (M. Krisper).

order and integration dependencies is addressed in the Scrum agile method with the definition of the role of a Product Owner who is in charge of overseeing them. Recent literature suggests several solutions for building up an explicit knowledge base for a project's integration dependencies. Lin et al. [9] proposed a method for decomposing complex processes into phased goals, and grouping low-level user stories with high-level goals. Clarke and Kautz [18] developed a method for factoring epics into user stories. Leffingwell [5] proposed a model which decomposes project's requirements in a tree view. Liskin [19] concludes that managing the granularity levels of user stories needs to be investigated more. Recent literature also discusses how to create an explicit knowledge base capturing the execution order dependencies among user stories. Milicic et al. [20] proposes a user-centric method which organizes user stories along scenarios and users. Leffingwell [5] suggests that the sequence of execution should be represented by use case diagrams and use case specification documents. Patton [21] proposes a story mapping technique for breaking big stories down while still maintaining the big picture of a project. The overall purpose of the mentioned solutions is to create a conceptual model that complements the list of user stories with information about the integration and execution order dependencies.

Conceptual models are key artifacts for understanding an application domain and its requirements [22]. Since most agile development projects aim to support business needs [23], it is essential to understand the processes of the business domain that the user stories are intended to support [24,25]. A process is a sequence (or a flow) of activities in an organization with the objective of carrying out work [26]. Processes can be represented as business process models. System development often makes use of process models [27,28]. These models bear the potential to explicate the integration and execution order dependencies among previously defined user stories; however, there is no prior research that investigates this potential.

This paper proposes and experimentally validates a novel method called BuPUS (Business Process User Story method), which associates user stories with business process model activities in such a way that execution order and integration dependencies can be easily traced. Similar to [28–31], we build on a theoretical framework for empirically evaluating the conceptual modeling techniques. We measure the understanding of a user story with problem-solving questions [29,32]. The results show that the understanding of the execution order and integration dependencies of a user story is greater when having the output of BuPUS available in comparison to only having a list of user stories.

We proceed as follows. We first discuss the importance of the knowledge about the execution order and integration dependencies of a user story for domain understanding. We review existing solutions for recognizing these dependencies. Next, we propose our BuPUS method that integrates user stories with BPMN models in order to enhance an individual's domain understanding. We then formalize our hypotheses with Mayer's theory of multi-media learning. For evaluation, we define an experiment design, show statistical results, and discuss threats to validity. Upon this basis, we highlight implications for research and practice. Finally, we conclude the paper and present directions for future work.

2. Background

2.1. Requirement dependencies

A requirement dependency refers to a situation in which the progress of the action of one user story assumes the timely outcome of the action of another user story or the fulfillment of a specific condition [10]. Dependencies are ubiquitous when developing an application and occur between people, groups, tasks, and artifacts, including the software components under construction [33]. They can cause problems such as bottlenecks, blockages in the flow of work,

and waiting, which potentially increase the likelihood of project delays, schedule overruns, and the need for task switching [34].

It is essential to understand requirement dependencies before carrying out the design of an application in order to avoid unnecessary implementation effort [35]. Focusing on dependencies as early as possible in a project is thus beneficial due to its potential to save rework and redesign [35]. On the other hand, ignoring the dependencies increases the risk attached to cost-effective project execution and, consequently, to project success [15,36–38]. Indeed, undetected dependencies can delay a project as people wait for resources, for the activities of others to be completed, or for necessary information [34].

Further, understanding requirement dependencies is of paramount importance for the successful deployment of agile development projects [35]. About 80% of all agile projects are supported by the agile development methods Scrum or XP [5]. Both of these popular methods suggest managing project requirements with user stories. This emphasizes the need to study dependencies among user stories (see [35,39,40]).

2.2. Dependencies among user stories

A user story is a brief statement that describes something the system needs to do for the user [5] and as such is business-oriented [41]. Its goal is to contain a few short notices about a desired function of the system [42] and provide a high-level overview of the requirements for a system [3]. Typically, a user story is created by following a general template [3,5,6]: *I as a <user role> can <function>*. As such, it is a non-visual requirement model. A user story model is typically used in agile software development projects [3]. A specific user story is just one of many user stories that represent a cohesive software product, and as such it cannot be treated as a stand-alone independent requirement. The dependencies among user stories need to be understood so that we can fully understand the domain of the software project.

The dependencies among user stories are a subject of speculation when previous knowledge about the domain is insufficient. We argue that the list of user stories alone hardly depicts the execution order and integration dependencies because of its three characteristics which we illustrate with user story examples in Table 1:

- 1) The naming of the user story functions can refer to different levels of abstraction (e.g. in Table 1 the function of user story US_299 “notify the customer about the credit committee's

Table 1
A list of user stories for Case 1.

List of user stories:	
US_189	I as a clerk in DEPT1 can modify the account's limit.
US_244	I as a clerk in DEPT1 can send a notification about a change in the account's attribute.
US_11	I as a clerk in the front office can create a request to modify the account.
US_299	I as a clerk in the front office can notify the customer about the credit committee's decision.
US_43	I as a clerk in the front office can give information about a rejection.
US_165	I as a clerk in the front office can read about the account's new limit.
US_99	I as a clerk in the front office can read about the decision.
US_999	I as a clerk in the front office can send a letter about the approval.
US_765	I as a credit committee (member) at a business unit can evaluate a non-standard account limit.
US_199	I as a credit committee (member) at a central unit can make an assessment report for a non-standard limit.
US_300	I as a clerk in the front office can notify the customer.
US_100	I as a credit committee (member) can evaluate the proposal.

decision" says nothing about what kind of decision was made or why the decision had to be made).

- 2) Different synonyms can refer to the same artifact (e.g. in Table 1 the following terms and phrases are used for the same artifact: the limit, the account's attribute, the non-standard account limit, the non-standard limit).
- 3) User stories can take part in different execution orders (e.g. US_100 and US_300 in Table 1 do not execute in the same business context as others do).

The development team can acquire information on dependencies among user stories from tacit and explicit sources of knowledge. In Scrum, a tacit source of knowledge is the Product Owner who is responsible for managing customer requirements [43,44]. On the other hand, conceptual models capture explicit knowledge about dependencies in order to promote understanding of the application domain, specifically about its requirements [22] and the dependencies among those requirements [10].

2.3. Explicit knowledge about execution order and integration dependencies

Agile development literature offers several solutions to build an explicit knowledge base regarding execution order dependencies among user stories. Leffingwell [5] suggests use cases (diagrams and specifications). He suggests dividing the main scenario of a user case into smaller chunks and associating user stories with the corresponding chunk. Ambler [3] suggests an approach with a level-0 data-flow diagram. Kulshreshtha et al. [11] depict relationships between requirements as nodes in the requirement network. Patton [21] proposes an approach called story mapping which is based on understanding user scenarios. The approach suggests placing physical user story cards on a timeline. In addition, Milicic et al. [20] propose a user-centric mapping method which organizes backlog items along scenarios and users by defining concepts of the domain and relations which exist between the concepts.

Some of the above-mentioned solutions serve as an explicit knowledge base about the integration dependencies among user stories. Patton's approach [21] suggests placing child user stories below large user stories to show a hierarchy. Leffingwell [5] proposes a so-called enterprise requirements model which groups user stories into features and features into epics which take part in a common project domain. Cohn [6] gives an overview of techniques for slicing one big user story up into smaller user stories. Altogether, the literature offers many solutions for managing the execution order and integration dependencies among user stories. However, empirical evidence on the effectiveness of these proposals is largely missing since the proposals are mainly motivated by practical experience [3,5,6,20,21].

2.4. Using business process models to better understand the dependencies

Business processes are essential for understanding a business domain which is intended to be supported by a new application [24,45,46]. Business process models are the most widely used type of conceptual models [47] and capture how the business works and how value is created for different business stakeholders [48]. The practice of business process modeling has emerged as a key instrument to enable decision-making in the context of analysis and design [24,49], process-aware enterprise systems [50], service-oriented architecture [51–53], and workflow operation [54].

A standard notation for modeling business processes is the Business Process Model and Notation (BPMN) [25]. Modeling projects which use BPMN are run for various reasons. If models are created for non-development purposes they are typically captured on higher levels of abstraction and focused on the analysis of the organization's

business processes (e.g. to meet the needs of an ISO certificate, to optimize the business processes using given resources) [24]. Since one of the agile principles is to take advantage of an organization's existing documentation [3], we further focus on the exploitation of existing higher-level business process models. Such models are typically available in both large [55] and in small- and medium-sized companies [56]. We aim to create an explicit knowledge base to increase understanding of the integration and execution order dependencies among user stories by exploiting existing BPMN business process models.

3. A novel method – integrating user stories and BPMN business process models

3.1. Introduction to creating the method

Method engineering is an approach to create software development methods that are customized to the specifics of organizations or projects [57]. A specific type of method engineering is situational method engineering which deals with adapting existing methods to specific project situations [58]. Situational method engineering approaches differ in several respects: the point of departure, the granularity of method fragments used to create situational methods, and the ability to execute. We use the Process Configuration Approach proposed by Bajec et al. [57]. This approach configures a new project-specific method by extending the base method. A base method encompasses instructions on how to handle various project types. It becomes richer every time new variations of handling projects are added. A new variation is added when a new project has a specific set of characteristics which cannot be correctly treated by any existing variation of the base method. Our proposed method called BuPUS is composed of a base and a project-specific method.

3.2. Generic data structure

As suggested by Saghaei and Wand [22], we use ontological concepts to clarify the meaning of the constructs of the method. In general, ontological concepts organize and describe what exists in reality in terms of: the properties, structure and interactions between real-world things [59]. Bunge's ontology and its concepts are the most widely used ontology in the analysis and design of information systems [29]. We use ontology's concepts to define the **constructs of a generic data structure**: a *thing*, *composite thing*, and *property* [60]. The world is made up of *things*. A *composite thing* consists of multiple *things* which are then referred to as components. A *property* can be either inherited (a *property* of a *composite thing* is also a *property* of a *component*) or emergent (a *property* of a *composite thing* is not part of any of its components). A *composite thing* must possess emergent *properties* of its own next to the inherited properties. These three constructs (shown in Fig. 1) are of the highest abstraction level (in later

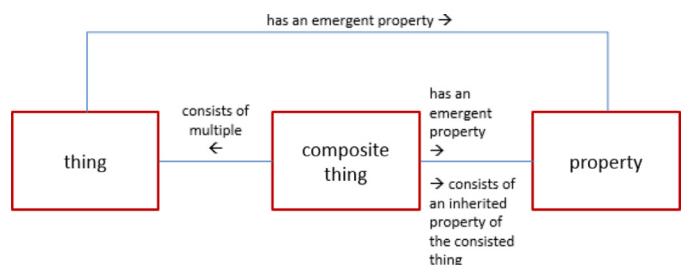


Fig. 1. Constructs of generic data structure.

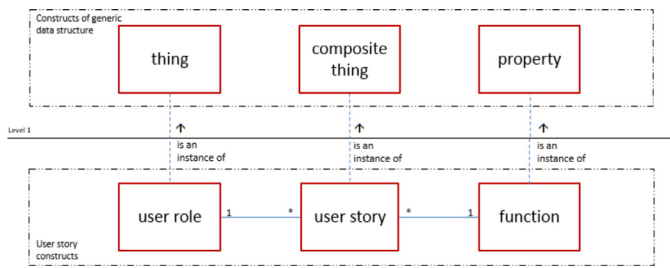


Fig. 2. Base method's data structure.

figures we refer to this level as level 1) and are used to derive more specific constructs of the base and the project-specific method.

3.3. Base method

The base method supports a simple technique for creating user story clauses, which is: *I as a <user role> can <function>*. In line with this template, the constructs *user role* and *function* are defined. *User role* is an instance of *thing*. Since a user is a specific person who interacts with software, we define *user role* as a generic term, which is common to multiple persons in the organization who execute the same set of functions. *Function* is an instance of *property* which represents a business activity that needs to be supported by the new application [5]. *User story*, as an instance of *composite thing*, consists of *function* and *user role*. The list of user stories can also include other information, such as estimates and priorities of user stories [6]. We have not included them in our base method since they are not relevant for understanding execution order or integration dependencies.

Fig. 2 shows the newly defined set of so-called **user story constructs** as instances of previously defined constructs of the generic data structure. The connection between constructs of a different level of abstraction is presented with a dotted line.

In general, a method includes (beside the data structure) a process for enacting it [61]. We do not specify a base method process because our project-specific process is meant to be applied to any of the existing agile development methods (e.g. Scrum, XP) which utilize user stories.

3.4. Project-specific method

3.4.1. Integration of the user story constructs and general business process constructs

In the following, we focus on projects with these characteristics: (1) a given list of user stories is available; (2) corresponding business process models are available; and (3) we want to understand the execution order and integration dependencies among user stories. In order to add a new variation to the base method, a new project-specific method has to be created [57]. We build the project-specific method in such a way that it can be used with any business process modeling notation. In this way, we leave flexibility for the future integration of other types of process models such as EPCs, UML Activity Diagrams, or Petri nets that all build on similar concepts [62]. In our paper, we focus on BPMN models.

A business process integrates information on what needs to be done, who is going to do it, and why [63]. In line with this definition, a set of so-called **general business process constructs** is depicted in Fig. 3. A *who* construct is an organizational role (e.g. a clerk in the front office), a *what* construct is an activity performed by the organizational role for a specific purpose, and a *why* construct is a reason for performing the business process. *Who*, *what*, and *why* are constructs on abstraction level 2 and are instances of constructs on abstraction level 1. The reasons for a particular derivation are discussed later, after we define the constructs on abstraction level 3.

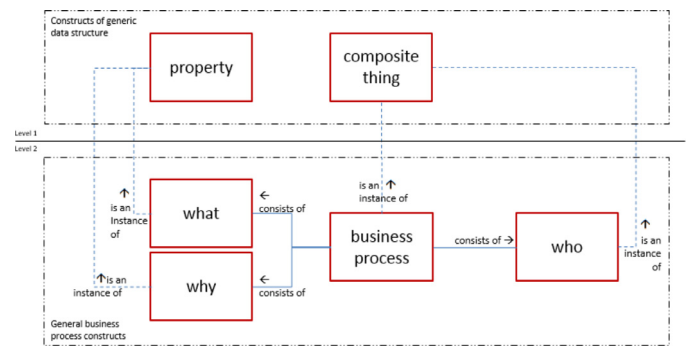


Fig. 3. General business process constructs.

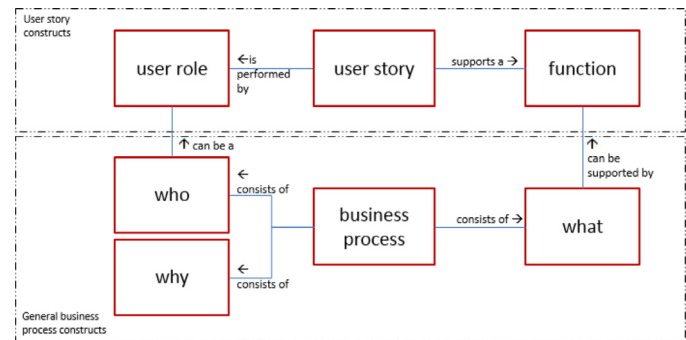


Fig. 4. An integration of the user story constructs and general business process constructs.

Figs. 2 and 3 provide the foundation for integrating user story constructs with general business process constructs. We integrate *function* (Fig. 2) with *what* (Fig. 3), and *user role* (Fig. 2) with *who* (Fig. 3). A specific business process can be partially or fully supported by a new application. Thus, when the function of a user story is intended to support a business activity, a connection between *what* and *function* is made. Similarly, *who* is connected with *user role* if the work of the organizational role is to be supported by at least one requirement of the new application. The integration is presented in Fig. 4.

With the constructs in Fig. 4 we cannot directly connect a specific pair of user story constructs (*user role* and *function*) with a specific pair of the general business process constructs (*who* and *what*) which is relevant for understanding the integration and execution order dependencies among user stories. Therefore, we decided to add constructs of a specific business process notation – BPMN. In addition, good user story conversations are also about the value of a function [21]. Conextra's user story template is used to this end: *I as a <user role> can <function> so that <added value>* [5,6]. The added value should help to distinguish important user stories from frivolous ones [6]. Nevertheless, the added value is easily understood as the *<business value I receive>* where *I* refers to the user role [5]. In some cases, several functions need to be executed so the user role can gain a meaningful value from the application. We argue that a user story should add value to the activities of a business process. Further, the output of the business process provides value to the internal or external customer of the organization [46]. Therefore, we aim to: 1) connect the two pairs discussed at the beginning of the paragraph by using BPMN constructs and by proposing new constructs; and 2) show the value of a user story as proposed above via the additional constructs.

3.4.2. Integration of the user story constructs and BPMN constructs

The BPMN grammar includes a plethora of elements, but in practice an average model uses only nine different BPMN elements [64]. For the integration, we need only frequently used BPMN elements, which we use to define a set of so-called **BPMN constructs**. The derived BPMN constructs represent abstraction level 3 and are related to abstraction level 1 via abstraction level 2.

Swimlane is a graphical container for partitioning a set of activities from other activities [26]. There are two specifications of a swimlane: a pool and a lane [26]. *Pool* represents a participant in a collaboration (e.g. a department). *Lane* is a partition that is used to organize and categorize activities within the pool (e.g. a specific organizational role within a department). Therefore, *swimlane* provides insight into who performs the specific activity. Like Recker et al. [65], we categorize *pool* and *lane* as two things which makes *swimlane* an instance of *composite thing*. Since *swimlane* is an instance of *who*, they are both instances of the same construct on abstraction level 1 – *composite thing*.

Activity can be either an atomic or non-atomic (compound) unit of work that an organization performs in its business processes [26]. Any *activity* consists of multiple *thing* constructs (discussed later in Section 3.4.3) and that makes it an instance of *composite thing*. In addition, it has two properties: *activity description* and *activity type*. The *activity type* construct specifies if the activity is atomic (the activity is not broken down into more detailed activities) or non-atomic (the activity has a collapsed process). *Activity description* is a clause which shortly describes the work of the organizational role. Since *activity description* represents an instance of *what*, they are both instances of the same construct on abstraction level 1 – *property*.

BPMN business process model is a set of activities that represent the steps required to achieve a business objective [26]. *BPMN business process model* has a property called *business process name*. *Busi-*

ness process name is an instance of the *why* construct. They are both instances of the same construct on abstraction level 1 – *property*. Further, the *BPMN business process model* consists of multiple *thing* constructs (e.g. *activity*) which makes it an instance of *composite thing*. Since *BPMN business process model* is an instance of *business process*, it is an instance of the same construct on abstraction level 1 – *composite thing*.

The use of the discussed BPMN constructs (represented in Fig. 5) facilitates the integration of a specific pair of BPMN constructs, namely *swimlane* (an instance of *who*) and *activity description* (an instance of *what*) with a specific pair of user story constructs, namely the *user role* and *function* constructs. We matched *user role* with *swimlane*, and *function* with *activity description*. Both *user role* and *swimlane* reflect organizational roles, such that they can be matched 1:1. On the other hand, matching *activity description* with *function* is not as simple since a granularity level of *function* in comparison to *activity description* needs to be defined. As suggested by Liskin et al. [19], we have different levels of a user story granularity. Therefore, we define new constructs: *association* (an instance of *thing*) and *granularity level* (an instance of *property*). So, when associating a user story with a BPMN activity element, the association is attributed according to one of the following user story association granularity levels:

- Level 1: the user story function is more abstract than the business process activity description (e.g. US_299 in Fig. 6);
- Level 2: the user story function is approximately equal to the business process activity description (e.g. US_99 in Fig. 6); or
- Level 3: the user story function is more detailed than the business process activity description (e.g. US_189 in Fig. 6).

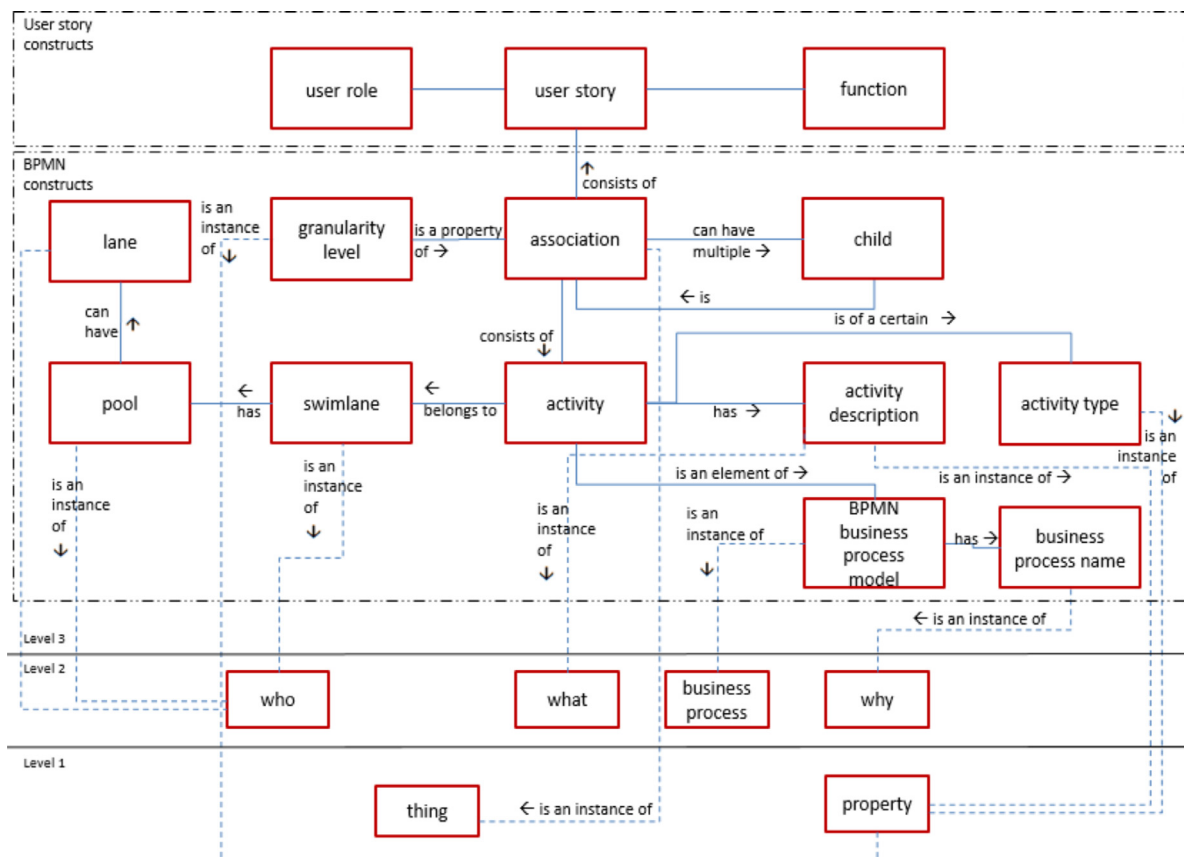


Fig. 5. An integration of the user story constructs and BPMN constructs.

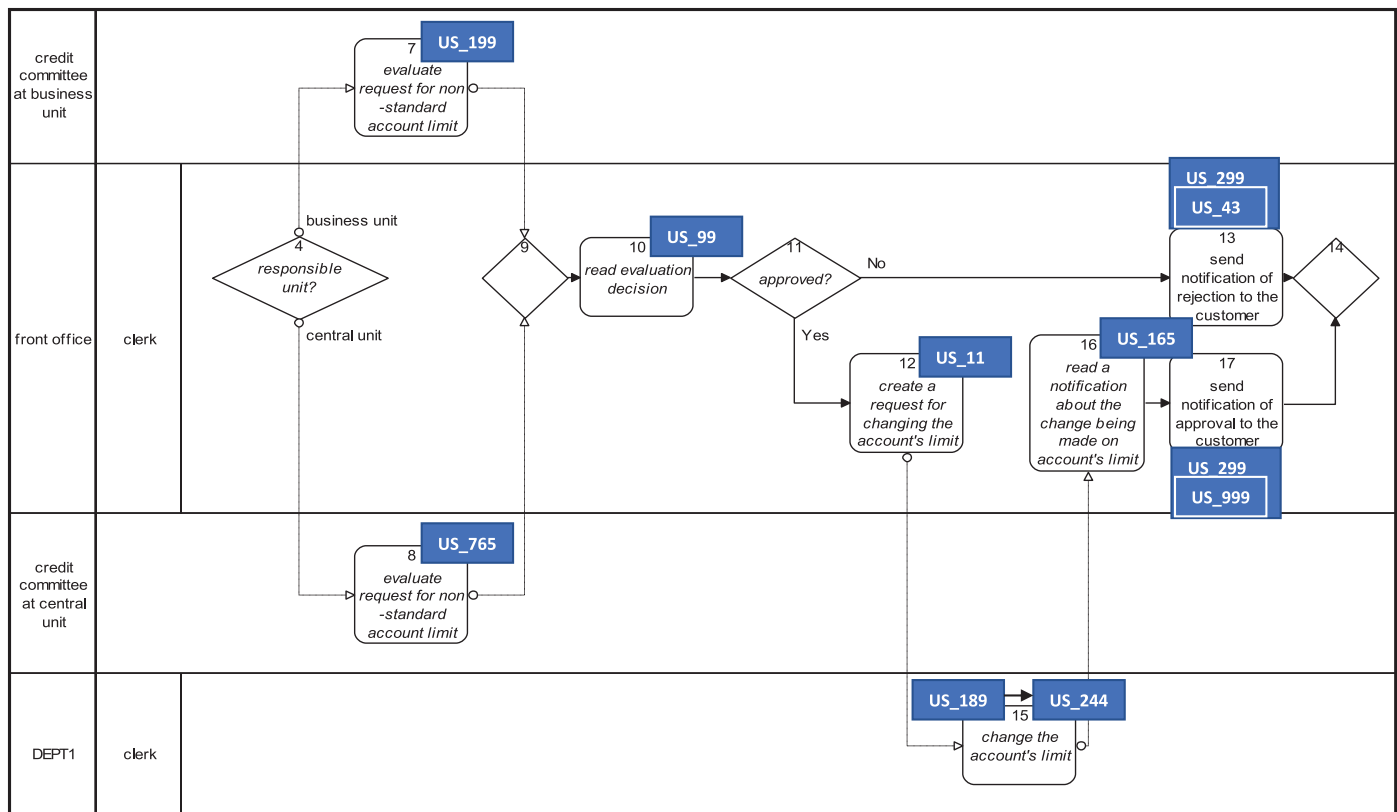


Fig. 6. Business process model with associations with user stories.

Fig. 5 depicts the integration of the user story constructs and the BPMN constructs. Only the BPMN constructs on level 3 are connected as instances of constructs on levels 2 and 1.

Fig. 6 depicts a business process model enriched with objects of user story associations. The objects are visually represented with a text-box object containing a user story code as a label.

When business process activities are associated with user stories, the value of a specific user story can be analyzed. For example, the function of user story US_244 from Table 1 is: “send a notification about a change in the account’s attribute”. As mentioned, the association of US_244 is a representative of an association at granularity level 3. The value of the function is to support the performance of a business activity “change the account’s limit” (see element number 15 in Fig. 6). Moreover, the function adds value to a business process called “getting a non-standard account limit” since it supports the process execution.

3.4.3. Execution order and integration dependencies

A user story can be influenced not just by a directly preceding user story but also by those which need to be executed before the previous one. This means that, to understand the execution order dependencies of a user story, we need to be aware of different execution orders which lead to the execution of a particular user story. Execution orders are managed by flow and gateway elements and, therefore, the set of BPMN constructs needs to be extended with additional constructs.

The flow element has two specializations: a sequence flow (an arrow with a full line) and a message flow (an arrow with a dotted line). A sequence flow element represents the sequence of activities within the same lane, and a message flow element represents the transmission of a message between two activities which are in different lanes [26]. Therefore, a construct called *flow* represents a sequence of activities while its property *flow type* shows to which of the two special-

izations a specific flow belongs. In addition, *flow* has two emergent properties: *start point BPMN element* and *end point BPMN element*. According to the modeling rules [66], each activity has exactly one input and one output flow. A new property called *input/output* shows if the flow is present on the input or on the output of a specific activity. As such, *flow* is a composite of *activity*.

Gateways define the flow between activities [26]. Therefore, a construct called *gateway* represents information about possible directions of the flow. A BPMN model can contain different specializations of a gateway, such as XOR [26]. According to the modeling rules [66], each split gateway should have a corresponding join gateway. We have defined three properties of *gateway*, namely, *gateway type* to characterize the specialization, *decision* to characterize the gateway decision description, and *split/join* to characterize the decision where necessary (namely, at split gateways). As such, *gateway* is an instance of *thing*, and also a composite of both *BPMN business process model* and *flow*. Since it is a composite of *flow*, this makes *flow* an instance of *composite thing*.

Here is an example of discovering execution orders that trigger the execution of US_299 (see Table 1). The business process model in Fig. 6 shows that the directly preceding user story can be either US_165 or US_11. US_165 is directly preceded by US_244 which is preceded by US_189. US_189 is directly preceded by US_11 which is directly preceded by either US_199 or US_765. If the preceded activity element is not associated with any user story, this indicates the end of the execution order. Therefore, the direct and indirect execution order dependencies of US_165 can be described by four different execution orders:

- 1) US_199 → US_11 → US_299
- 2) US_765 → US_11 → US_299
- 3) US_199 → US_11 → US_189 → US_244 → US_165 → US_299
- 4) US_765 → US_11 → US_189 → US_244 → US_165 → US_299

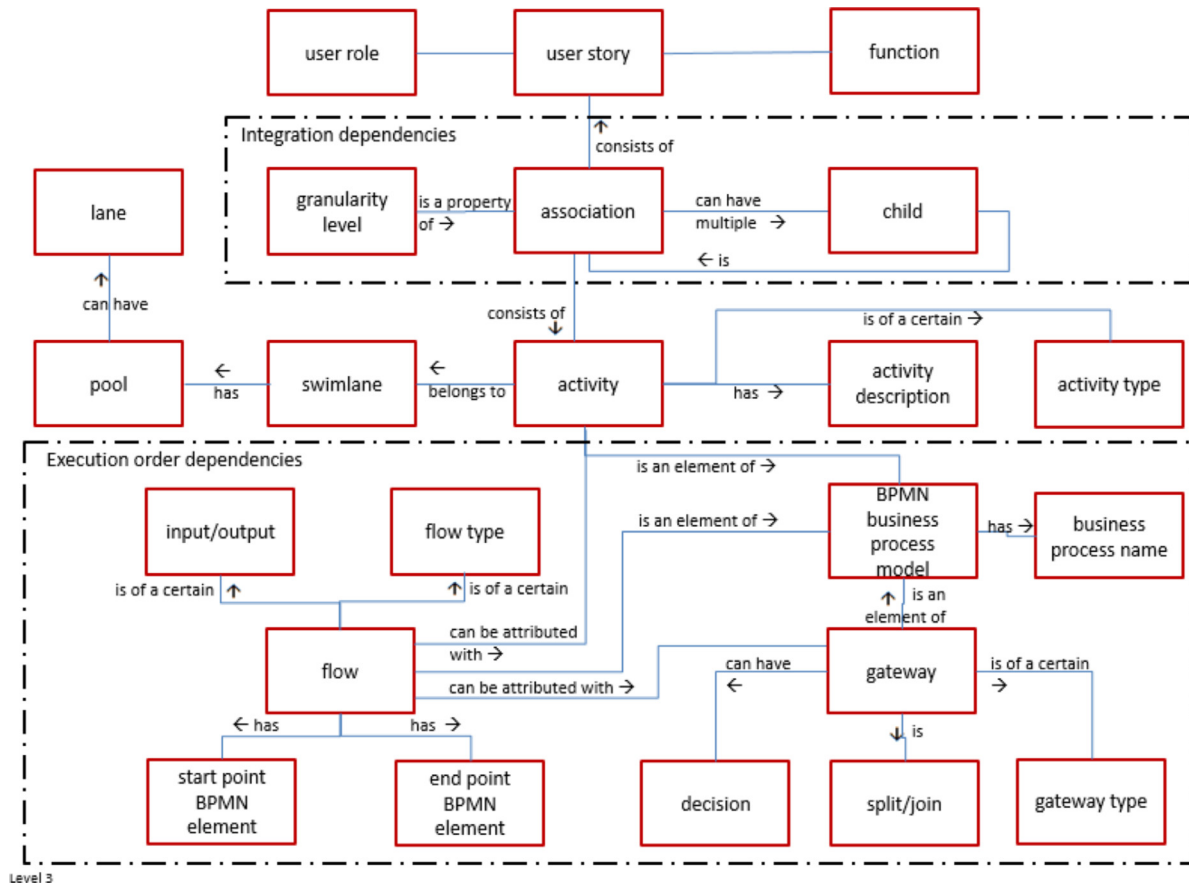


Fig. 7. Constructs for managing execution order and integration dependencies.

On the other hand, understanding integration dependencies requires information about the hierarchy relation of the user stories on the list; for example, which user story represents a composite of a higher-level (more abstract) user story. We can visually represent the composition in many ways. Fig. 6 shows a nesting of association objects technique where the more detailed user story association object is visually smaller and within the bigger, more abstract user story association object. US_299 is more abstract than US_43 and US_999. The two user stories represent specific execution variations of US_299. For managing the integration dependencies, we suggest using two constructs that we have already defined, namely, *association* and *granularity level*, and an additional property called *child*. This last property makes a note about user stories that compose a specific user story. Fig. 7 depicts the constructs.

Fig. 7 shows a whole set of a **BuPUS constructs** at abstraction level 3. The constructs are grouped according to their relevance for understanding integration and execution order dependencies among user stories.

An overview of how the constructs on abstraction level 3 derive from the constructs on abstraction levels 2 and 1 is presented in Table 2.

3.4.4. The BuPUS process

Fig. 8 shows how to prepare the material needed for a better understanding of the execution order and integration dependencies of a user story according to our BuPUS. As an input for the first activity “select relevant business process models”, we need a list of user stories and the business process models. Since companies often have many business process models explicitly defined, we need to select only those models that are relevant for the application domain. Then,

we need to “modify the models to meet the modeling guidelines” as suggested by Mendling et al. [66] because not all process models in practice are of excellent quality [67]. Then, we “associate user stories with (corresponding) activity elements”. Afterwards, we need to “identify the activity elements with multiple user story associations” in order to be able to “structure the user stories within the same activity element”. To visually represent the structure we use the nesting of association objects technique. The outcome of BuPUS is a script consisting of the list of user stories which is accompanied by associated business process models. An example of how we followed this process when preparing material for Case 1 is presented in Section 5.5.

4. Hypotheses

We argue that the method presented in Section 3 improves the understanding of the execution order and integration dependencies of a user story. Understanding of the user story dependencies is measured with two different types of problem-solving questions where one requires understanding of the execution order dependencies and the other of the integration dependencies between given user stories (for more details, see Section 5.4). Consequently, we formulated two propositions:

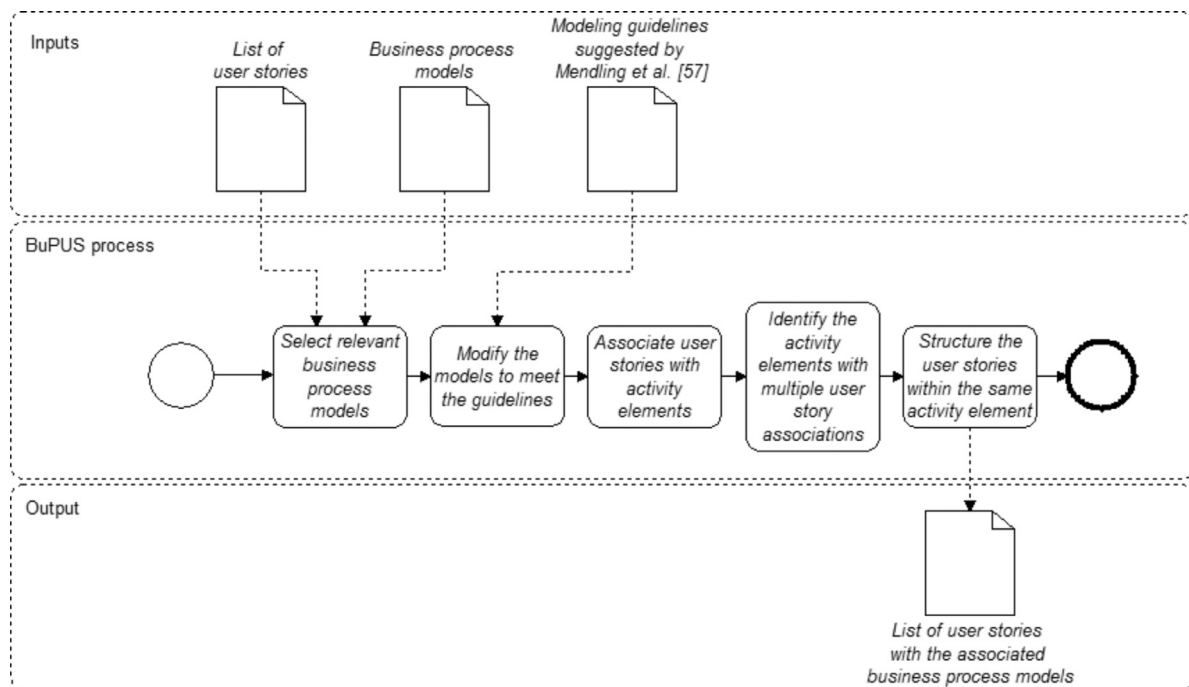
Proposition 1. *Understanding of the execution order dependencies among user stories is greater when using the BuPUS script in comparison to reading the list of user stories alone.*

Proposition 2. *Understanding of the integration dependencies among user stories is greater when using the BuPUS script in comparison to reading the list of user stories alone.*

Table 2

An overview of the BuPUS constructs and their derivation relationships.

Generic data structure's constructs (level 1)	Constructs from level 2	Constructs from level 3
thing	–	user role
	–	association
	–	gateway
composite thing	–	flow
	who	swimlane
	–	activity
	business process	BPMN business process model
	–	user story
property	why	business process name
	–	granularity level
	what	activity description
	–	activity type
	–	flow type
	–	gateway type
	–	split/join
	–	decision
	–	start point BPMN element
	–	end point BPMN element
	–	input/output
	–	child
	–	function

**Fig. 8.** BuPUS process.

4.1. Theory

In order to decompose the propositions into hypotheses, we discuss cognitive theories. The focus on cognitive theory is natural when considering conceptual modeling techniques [68]. We follow recent work [22,25,29] when discussing Mayer's cognitive theory of multi-media learning [32]. The theory has frequently been used to evaluate conceptual modeling grammars [22]. It is a theory of how people learn from words and pictures. Mayer's research relies on the experimental comparison in which an experimental group of learners receives a lesson that contains the to-be-tested feature while a control group of learners receives an otherwise identical lesson that lacks the to-be-tested feature. Subsequently, both groups take a problem-solving test.

Extraneous processing is cognitive processing during learning that does not serve the learning goal and causes a cognitive overload [32]. The list of user stories has three characteristics which cause extraneous processing. First, it contains user story clauses which are from different business process models. Learning the execution order and integration dependencies from such a list causes extraneous processing. Second, the user story clauses use synonyms for the same thing. An assumption of our study is that no glossary is available. Third, the user stories are from different levels of abstraction. These three characteristics, which decrease the understanding of the dependencies among user stories, can be present in real-life projects. We took these characteristics into account when preparing the material for our empirical evaluation.

Ottenssooser et al. [25] demonstrate that when using textual and graphical notation at the same time the performance of domain

understanding is the highest. BuPUS adheres to the spatial contiguity principle of Mayer's theory which suggests placing essential words next to the corresponding graphical element. Specifically, BuPUS places the user story code labels next to the corresponding BPMN activity element. By associating user stories to activity elements, a clear representation of the execution order of the user stories is gained by following BPMN flow arrows. Since the activity elements follow a temporal sequence, the user stories associated to those activities also need to follow that sequence. The BuPUS output consists of words and pictures, while the list of user stories only has words. We believe this is one of the reasons that the BuPUS script should provide sufficient support in understanding the dependencies a user stories.

The expressiveness of a grammar is its ability to generate scripts that capture information about a modeled domain [68]. Wand and Weber [69] propose evaluating modeling methods by analyzing their constructs. In Section 3, we discussed the expressiveness of two grammars, namely, the BuPUS grammar, and the grammar used by the list of user stories, such that we can examine their ability to convey information about the execution order and integration dependencies. In our research, the grammar used by the list of user stories is not different from the grammar that is used for generating one single user story clause. This kind of list of user stories is commonly found in a phase of project preparation which occurs before the first development iteration. The user story template is simple since the customer side of the development project is responsible for delivering its list of user stories [6]. Consequently, the user story clauses are too coarse to promote understanding of the dependencies [19]. Later, in development iterations the knowledge about specific user stories grows with the communication between the development team and the customer representative [6]. Fig. 2 shows the state of user stories in the project phase of preparation where there are no constructs holding information about the execution order or integration dependencies among user stories. In BuPUS we obtain information about the execution order dependencies among user stories in an explicit way while being influenced by Milicic et al. [20] and Leffingwell [5]. While they proposed the creation of certain scripts, we suggest reusing existing documentation – the business process models. With BuPUS the information about the execution order dependencies among user stories can be read from the model by following the flow arrows. We also focused on the integration dependencies among user stories. Similar to Patton [21], Lin et al. [9], Clarke and Kautz [18] and Leffingwell [5], we group user stories of different abstraction levels together and depict them in a specific structure with a technique we call nesting of association objects. Fig. 7 depicts the BuPUS constructs that are used to hold information about the execution order and integration dependencies among user stories. We believe that the BuPUS script is sufficiently grammatically expressive to share explicit knowledge about the execution order and integration dependencies among user stories with the project's stakeholders since there are constructs which capture that information.

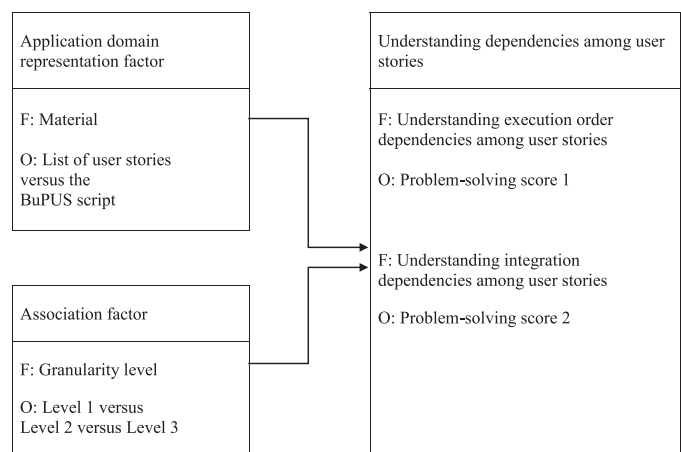
When comparing material for two experimental treatments while using two different grammars, the results might be confounded because one model might contain more information than the other or one may be easier to understand than the other [68]. As suggested by Gemino and Wand [68], we discuss this issue by analyzing informational equivalence. The term informational equivalent means that all information in one material is inferable from the other and vice versa [68,70]. In our experiment, the control group was given the list of user story clauses (see the example in Table 1), while the experimental group was given the list of user story clauses as well as the associated business process model (see the example in Fig. 6). When preparing the material, we strove to move close to informational equivalence. First, the set of user roles discussed on the list is the same as the set of BPMN swimlanes (e.g. user role “clerk in the front office” and pool “front office” with the lane “clerk”). Second, the functions of user sto-

ries are associated with activity elements where no activity element is left without a user story association. The difference in materials is in the additional elements of the associated business process model, namely, the decision and merge XOR gateways, and flow arrows. We use flow arrows to show the sequence of activity elements and consequently the execution order dependencies among user stories. We use XOR gateways to demonstrate how the integration dependencies among user stories can be shown in an associated business process model.

Next, when associating user stories with activity elements we would move closer to information equivalence if each association object was linked to exactly one user story. But since we wanted to investigate three different types of association granularity (namely, a user story is: (a) approximately equally big as; (b) more abstract than; or (c) more detailed than its corresponding activity element), the association labels on the business process models were not equally distributed to the activity elements. As a result, some activities had more than one association and some user stories were associated to multiple activity elements.

4.2. Research model

Fig. 9 shows our research model. Two factors have an influence on an understanding of the dependencies: an application domain representation factor and an association factor. First, the application domain is represented by one of the two materials: the list of user stories or the BuPUS script. Second, each user story association is characterized by one of the three granularity levels: levels 1, level 2, or level 3. If they were modeled in an abstract manner compared to the associated activity description, we would obtain level 1 user story associations. If they were modeled at a similar abstraction level as the user stories, we would obtain a level 2 user story association. And if the given business process models activity elements were modeled in great detail compared to the associated activity description, we would obtain a level 3 user story association. With this setting, we can compare how each of the two application domain representations of the three granularity levels influences the understanding of the execution order and integration dependencies among user stories. From Proposition 1 which focuses on the understanding of the execution order dependencies, we created three hypotheses for each user story granularity level: H1, H2, and H3. Similarly, from our Proposition 2 which focuses on the integration dependencies we created hypotheses H4, H5, and H6.



Key: F = theoretical factor, O = operationalization factor

Fig. 9. Research model.

1. H1: When a user story is more abstract than the corresponding activity in a business process model, the understanding of the *execution order dependencies* among user stories is greater using the BuPUS script than by reading the list of user stories alone.
2. H2: When a user story is approximately equally big as the corresponding activity in a business process model, the understanding of the *execution order dependencies* among user stories is greater using the BuPUS script than by reading the list of user stories alone.
3. H3: When a user story is more detailed than the corresponding activity in a business process model, the understanding of the *execution order dependencies* among user stories is greater using the BuPUS script than by reading the list of user stories alone.
4. H4: When a user story is more abstract than the corresponding activity in a business process model, the understanding of the *integration dependencies* among user stories is greater using the BuPUS script than by reading the list of user stories alone.
5. H5: When a user story is approximately equally big as the corresponding activity in a business process model, the understanding of the *integration dependencies* among user stories is greater using the BuPUS script than by reading the list of user stories alone.
6. H6: When a user story is more detailed than the corresponding activity in a business process model, the understanding of the *integration dependencies* among user stories is greater using the BuPUS script in comparison to reading the list of user stories alone.

5. Research method

5.1. Research design

The research design is structured as suggested by Gemino and Wand's framework for the empirical evaluation of conceptual modeling techniques [68]. Since we are comparing two different grammars, namely the list of user stories versus the BuPUS grammar, we have an inter-grammar comparison. Rules within the grammar were not varied. The medium of presentation was paper using text and graphics and was not varied between treatments. The task was to interpret the given material and the subjects were novice modelers. The focus was placed on the evaluation of the cognitive model created by viewing the given material. Therefore, the product of script interpretation that we measure is understanding of a user story. Product effectiveness was measured in several ways using comprehension, problem-solving (transfer) and recall tests. In addition, the product efficiency was assessed using questions to measure the perceived ease of use and usefulness.

5.2. Experimental design

We decided to conduct an experiment, which is a research method promoted in the design science research methodology [71]. We used a one-factor experiment with two treatments with a completely randomized design. We conducted several pilot experiments between January and March 2015 in order to test the performance of a subject in understanding the BuPUS script. During the pilot phase, we made sure that the questionnaire was comprehensible and possible to be completed in 50 min. The feedback gained from the pilot experiments also influenced the BuPUS development (e.g. the set of constructs that we presented as relevant). The experiment was conducted in April 2015.

For the experiment we created two workbooks, namely workbook 1 and workbook 2 as depicted in Table 3. In both workbooks, the first treatment is *guided* and refers to having an additional corresponding business process model associated with the user stories. The second treatment is *unguided* and refers to having solely the list of user stories available. Each participant was exposed to both treatments from

Table 3

A composition of workbooks.

Workbook	Treatment 1: guided	Treatment 2: unguided
Workbook 1	Case 1	Case 2
Workbook 2	Case 2	Case 1

two cases, namely, Case 1 and Case 2, which were both from the banking sector. Data collected from the guided and the unguided treatment of the same subject were treated independently. Therefore, the method of data collection used was a between-subject design.

The task for our subjects was to interpret the given material. This involved asking subjects to read the material provided and to give answers to the questions about user stories. The questions were focused on measuring understanding of the user stories. Mayer [32] promotes problem-solving tests to evaluate how well the learner understands the material. A measurement instrument that Mayer uses in his experiments is a questionnaire and the measures are its scores. As suggested by Gemino and Wand [68], we measured the subjects' performance in comprehension, problem-solving and recall tests.

The first test for each treatment was a comprehension test on the given material. The test was used for control purposes since we wanted to ensure that the subjects made an effort and developed their cognitive model. Afterwards, the subjects took the problem-solving test. The problem-solving questions were used to support the hypotheses. Next, a second comprehension test was conducted, but this time with material removed (called a recall test in the continuation). The recall test was also used for control purposes only. The last set of questions for each treatment included self-assessment items. They were about the (perceived) ease of use and usefulness of the given material.

5.3. Subjects

The subjects in our experiment were 127 undergraduate students attending the Business Information Systems II course at Vienna University of Economics and Business in Austria. Each student was attending one of seven classes. The classes were managed by six different lecturers. Given that the students had learned about business process modeling before, that the understanding of the concept of a user story is relatively simple, and that knowledge about the guided case was given on the spot, the subjects in our experiment were not required to be experts in either BPMN modeling or agile development with user stories. The students were familiar with another modeling notation so reading simple BPMN models was no trouble for them. Further, they were informed about the INVEST qualities that should be met by a user story, and about our assumption that all the user stories in the given material meet those qualities. The selected students are likely to be good enough representatives of development project stakeholders who want to communicate about the execution order and integration dependencies among defined user stories [72].

Background questions were asked about gender, nationality, age, perceived knowledge of the English language and familiarity with user stories, business process models, BPMN and bank-specific processes. The group included 85 male and 41 female students while one subject did not specify his or her gender. More than 90% of the students were under the age of 26. Our experiment was in English but less than 1% of our students were native English speakers. Nevertheless, 90.5% marked their perceived familiarity with the English language on a Likert scale (1=poor, 7=excellent) with a grade of 4 or more. Moreover, 37.8% of students had never heard about user stories before (Likert scale: 1=not at all familiar, 7=very familiar) while 58.3% marked grades 2 to 4, respectively. Further, 85% of the students marked their perceived familiarity with business process models with a grade of 4 and less (Likert scale: 1=poor, 7=excellent). In

addition, 42.5% of the students had never learned about the BPMN notation before, while another 48.8% of the students graded their knowledge with scores between 2 to 4 (Likert scale: 1=poor, 7=excellent). Moreover, 78.7% of the students scored their perceived familiarity with the specific business process “getting a new MasterCard” with 4 or less (Likert scale: 1=not at all familiar, 7=very familiar), and 92.9% of the students gave a grade of 4 or less (Likert scale: 1=not at all familiar, 7=very familiar) for perceived familiarity with the business process “getting a non-standard account limit”.

5.4. Questionnaire

5.4.1. Comprehension test

By scoring a comprehension test we measure how well a subject can read the given material. The test questions can be answered by either looking at the list of user stories or looking at the BuPUS product. In the associated business process model, the user story dependencies can be visually interpreted while from the list of user stories they can only be textually interpreted. We used the following questions:

- Which user role executes the user story US_*? (radio buttons)
- What are the two “card limit types” that the customer can ask for? (radio buttons)
- What are the two “customer types”? (open question)
- Is the execution of US_* optional? (Yes / No / Do not know)
- Which user stories have the potential to be executed when the bank’s customer wants to get a new MasterCard? (check boxes)
- Which user roles need to collaborate when the bank’s customer wants to get a new MasterCard? (check boxes)

5.4.2. Problem-solving test

The problem-solving test consists of two types of questions where one type evaluates the understanding of execution order dependencies and the other of integration dependencies. Each type has its own template. The first problem-solving template question focuses on the understanding of the execution order of a specific set of user stories.

Template 1. *What was previously done wrong (or not done at all) so the activity <specific user story function> cannot get started?*

We narrowed down the possible answers by providing additional guidance. We broke question Template 1 down into three specific sub-questions. As shown in Fig. 10, under sub-question: (a) the subject needs to fill in the user story codes about the execution order (with the subjects we used a word *sequence*) of user stories which can lead to the execution of user story US_299. The subjects can recognize that there are four different execution orders from the question itself. Subjects with guided material are able to see the execution orders by following the flow arrows between the associated business process activities. On the other hand, subjects with unguided material need to read the user story clauses and understand the execution orders from that. Next, sub-question (b) refers to the execution orders of (a). In (b) the subjects need to find out where in the execution order the work is passed to another user role. The answer requires to check if the first and the second user story in the execution order have different user roles, then the second and the third, then the third and the fourth and so forth. Subjects with guided material could see the shift by observing flow arrows which cross over to another swimlane, while subjects with unguided material need to read the user story clauses and understand the information from that. Lastly, sub-question (c) also refers to the execution orders of (a). The subjects need to understand if there are any business decisions taken which influenced the further execution order. Subjects with guided material could recognize the decision points by understanding the XOR gateway elements, while subjects with unguided material need to understand these matters by reading the user story clauses. Points

a) Fill in the gaps with the user story codes so we can see all the variations of how the execution of US_299 can be approached.
 Sequence 1: US ____ -> US ____ -> US ____ -> US ____ -> US ____ -> US ____ -> US_299
 Sequence 2: US ____ -> US ____ -> US ____ -> US ____ -> US ____ -> US ____ -> US_299
 Sequence 3: US ____ -> US ____ -> US_299
 Sequence 4: US ____ -> US ____ -> US_299

b) Between which two user stories in the sequences of a) does a change of user roles happen?
☐ I do not know if there are any pairs.
☐ I am certain that there are no pairs.
☐ There are the following pairs:
 US ____ & US ____; _____

c) Are there any decision points in the sequences of a) which influence the variability of execution sequences?
☐ I do not know.
☐ No, I do not believe there are any decision points.
☐ Yes.

If Yes a mistake was made at a decision point so the incorrect following flow (the alternative flow) of user stories was selected. The US_299 is desperately waiting for (correct) input information. What could have gone wrong at previous decision point(s) (please tick)?
☐ Maybe the evaluation was made by wrong credit committee.
☐ Maybe the evaluation was not approved but treated as approved.
☐ Maybe the evaluation was approved but treated as not approved.

Fig. 10. Example of Case 1 sub-questions used for answering a Template 1 question (caption from a workbook).

are given for each correctly filled in gap, for choosing the right radio button and for ticking the correct check boxes.

For example, in Case 1 there is a problem-solving question: *What was previously done wrong (or not done at all) so the activity “send a notification about a change in the account’s attribute (US_244)” cannot get started?* The answer to a Template 1 question is a combination of answers to sub-questions a), b), and c). Therefore, the answer to the question refers to Fig. 10 and is composed like this:

- Maybe one of the preceding user stories was not executed correctly – see the user stories listed under a).
- Maybe there was a communication failure between two user stories when passing information from one user role to another – see the pairs of user stories listed under b).
- Maybe a wrong decision about a sub-flow was made – see the boxes under c) which are ticked.

The second problem-solving template question requires understanding of the integration dependencies of a specific user story.

Template 2. *Sometimes the activity <specific user story activity> has to be executed in a different way. When does that happen?*

To answer the Template 2 question, the composite user stories of a particular user story need to be available on the list of user stories. An example question is: *Sometimes the activity “notify the customer about the credit committee’s decision (activity from US_299)” can be executed in different variations. Which user stories (codes) present those variations?* Fig. 6 shows the nesting of association objects: an association object with label US_299 (I as a clerk in the front office can notify the customer about the credit committee’s decision) has two specializations: US_43 (I as a clerk in the front office can give information about the rejection) and US_999 (I as a clerk in the front office can send a letter about the approval). The answer to the question in Fig. 11 consists of choosing the second radio button and filling in the keywords on the line: rejection (or US_43) and approval (or US_999).

Sometimes the activity “notify the customer about the credit committee’s decision (activity from US_299)” can be executed in different variations. Which user stories (codes) present those variations?

- There are no user stories presenting the wanted variations.
- The following user stories are presenting the variations:

Fig. 11. Example of a Case 1 question generated with a Template 2 question (caption from a workbook).

Table 4

Problem-solving questions of Case 1 and their relationship to the hypotheses.

Question	Hypothesis
Sub-questions of Template 1 addressed to US_299 (see Fig. 10)	H1
Sub-questions of Template 1 addressed to US_11	H2
Sub-questions of Template 1 addressed to US_244	H3
Template 2 addressed to US_299 (see Fig. 11)	H4
Template 2 addressed to US_11	H5
Template 2 addressed to US_244	H6

Each problem-solving template question is used three times per case. The three times applies to three situations when a user story is: (1) more abstract than; (2) approximately equally big as; and (3) more detailed than the corresponding activity in a business process model. Table 4 presents an overview of the questions for Case 1 and their relationship to the hypotheses.

5.4.3. Recall test

The recall test measures how much information about the domain has remained in the memory of the subjects (see e.g. in Fig. 12).

5.4.4. Ease of use and usefulness

Questions from Maes and Poels [73] are used to measure ease of use and usefulness on a 7-point Likert scale with response options ranging from “strongly disagree” to “strongly agree”. The questions are asked separately for the guided treatment and the unguided treatment. The statements were:

A credit committee needs to evaluate a request for a non-standard account _____. There are two kinds of credit committee. One kind of credit committee is formed at _____ unit and another is formed locally at _____ units. Which credit committee will be dealing with the customer’s request depends on a specific credit committee’s responsibilities. After the ‘evaluation _____’ is made, it is forwarded to a clerk in the front office. If it is positive, then the clerk in the front office fills in a _____ form for changing the ‘account _____’. When the change is executed, the clerk in the front office informs the _____ about it by writing a notification of _____.

Fig. 12. Recall tasks for Case 1 (caption from a workbook).

- It was easy for me to understand what the material was trying to show.
- Using the material was often frustrating.
- Overall, the material was easy to use.
- Learning how to read the material was easy.

5.5. Materials

In our design, we used business process models which were developed for a European bank for analyzing and optimizing the business and not for IT development. We chose an ‘account management’ topic, which had 24 business process models available. Out of those 24 models, we chose two of them for our experiment: “getting a new MasterCard”, and “getting a non-standard account limit”. We made modifications to the two models by following the seven process modeling guidelines [66]: we consistently used message flow arrows when the following activity was in another pool, we added merge gateways to every XOR decision gateway, and used verb-object activity labels. When making these modifications, we were careful not to change the information of the business process models but only its visual presentation.

The BPMN models were created in iGrafx. For our research, the activity element is the most important element since it is associated with a user story. Awareness of its surrounding elements such as swimlanes, XOR gateways, and flow arrows is also important for understanding the dependencies. For each case, we provided a business process model with these elements. An example of a business process model for Case 1 is shown in Fig. 6.

After the business process models were prepared, we created two lists of user stories. Our assumption is that all user stories on the list meet the INVEST quality aspects and that they are selected to enter in a common development iteration. An example of the list of user stories for Case 1 is shown in Table 1. In our material we incorporated the three characteristics of the list of user stories discussed in Section 2.2. The first characteristic is “the naming of the user story functions can refer to different levels of abstraction”. We associated the function to a business process activity element with one of the three levels of the association granularity in the following way:

- Level 1: A situation when a user story is more abstract than the corresponding activity in a business process model. This was done by describing the user story function on a higher level of abstraction than the associated activity element description. An example of a level 1 association is shown in Fig. 6 where US_299 is associated to multiple activity elements.
- Level 2: A situation where a user story is approximately of equal abstraction as the corresponding activity in a business process model. This was done by describing the user story function and activity element description in a very similar way or exactly in the same way. An example of a level 2 association is shown in Fig. 6 where US_11 is included.
- Level 3: A situation where a user story is more detailed than the corresponding activity in a business process model. This was done by presenting a user story function as a very detailed task of the associated activity element. An example of a level 3 association is shown in Fig. 6 where multiple user stories (US_189 and US_244) are associated to one activity element.

The second characteristic “different synonyms can refer to the same artifact” is integrated into our material as described in Section 2.2. And the third characteristic “User stories can take part in different execution orders” means that not all user stories from the list of user stories in Table 1 (see US_300 and US_100) are associated to the activities of business process model presented in Fig. 6.

For preparing an associated business process model for Case 2, we followed the specific set of rules described in Table 5.

Table 5
Rules for generating additional cases.

Association level	Association level's characteristic
Level 1	The user story is associated to multiple activity elements. AND The associated activity elements are between the same XOR decision and XOR merge gateway. AND The associated activity elements are each in their own parallel alternative path.
Level 2	The user story is associated to one activity element. AND The user story function is relatively equally big as an activity element.
Level 3	The user story is associated to one activity element. AND The activity element has multiple associations. AND The user story function is more detailed than the associated activity element. AND The sequence of user stories associated to the same activity element is easily recognizable.

Table 6
Experimental protocol.

Step	Description of the step	Timing
1.	The researcher explained the purpose, structure and rules of the experiment.	App. 5 min
2.	Students solved questions about their background.	Max. 2 min
3.	The researcher gave an oral explanation of the example material.	App. 7 min
4.	The students solved the comprehension test of the guided treatment's case.	Max. 7 min
5.	The researcher gave a verbal explanation of the example questions.	App. 7 min
6.	Students solved the problem-solving test of the guided treatment's case.	Max. 12 min
7.	Students solved the recall test of the guided treatment's case.	Max. 5 min
8.	Students solved questions about the perceived ease of use and usefulness of the guided treatment's case.	Max. 5 min
9.	The students solved the comprehension test of the unguided treatment's case.	Max. 7 min
10.	Students solved the problem-solving test of the unguided treatment's case.	Max. 12 min
11.	Students solved the recall test of the guided treatment's case.	Max. 5 min
12.	Students solved questions about the perceived ease of use and usefulness of the guided treatment's case.	Max. 5 min

Table 7
Treatments and the number of subjects.

Case	Treatment 1: guided	Treatment 2: unguided
Case 1	63	64
Case 2	64	63
Number of subjects	127	127

5.6. Operation

The workbooks were printed in landscape orientation on both sides. In this way, two pages were seen at the same time (e.g. pages 2 and 3, pages 4 and 5, etc.). The page on the left-hand side was used to present the material. The workbooks were given to the students so that two students who were neighbors were not solving the same workbook. We gave them instructions not to flip forward unless they had been told to do so and to work from the beginning through to the end of the workbook. They were not allowed to flip the page back.

The experiment was executed in approximately 70 min, 50 min of which was used for solving the workbooks. Three students returned their material untouched. Table 6 presents the experimental protocol.

Table 7 shows the number of students who solved the workbooks for each case and treatment.

6. Results

6.1. Descriptive statistics and normality testing

As Table 8 shows, the maximum scores of Case 1 and Case 2 were not equal. That is why we normalized scores for each of the problem-solving answers, the sum of comprehension answers, and the sum of recall master answers to 100 for both Case 1 and Case 2.

Table 9 shows descriptive statistics for the problem-solving questions generated with Template 1 for both guided and unguided treatment. To provide answers to the questions generated with Template 1, the subjects needed to have an understanding of the execution order dependencies. We generated three problem-solving questions which were set for three different associations: first, for the association characterized with granularity level 1 (question Template1_ABSTRus), second, for associations characterized with granularity level 2 (question Template1_EQUALus) and third, for associations characterized with granularity level 3 (question Template1_DETAILus). For the guided treatment, the mean score for Template1_ABSTRus is 74.60 and for the unguided treatment it is 18.14. The mean score for Template1_EQUALus is 71.92 for the guided treatment and 12.50 for the unguided treatment. Finally, the mean score for the Template1_DETAILus is 74.28 for the guided treatment and 18.74 for the unguided treatment.

Similarly, Table 10 shows descriptive statistics for measurements taken for the problem-solving questions generated with Template 2. To provide answers to the questions generated with Template 2, we need to have an understanding of the integration dependencies. With Template 2, we generated three problem-solving questions which were set for three different associations: first, for the association characterized with granularity level 1 (question Template1_ABSTRus), second, for associations characterized with granularity level 2 (question Template1_EQUALus) and, third, for associations characterized with granularity level 3 (question Template2_DETAILus). For the guided treatment the mean score for Template2_ABSTRus is 64.90 and for the unguided treatment it is 21.00. The mean score for Template2_EQUALus is 64.57 for the guided treatment and 47.24 for the unguided treatment. Finally, the mean score for Template2_DETAILus is 74.80 for the guided treatment and 49.61 for the unguided treatment.

Table 8
Original and recalculated maximum scores of questions.

Variable	Case 1 original max score	Case 2 original max score	Recalculated max score used for statistical analysis
Template1_ABSTRus	15	3	100
Template1_EQUALus	10	9	100
Template1_DETAILus	25	10	100
Template2_ABSTRus	3	4	100
Template2_EQUALus	1	1	100
Template2_DETAILus	1	1	100
Comprehension_sum	20	16	100
Recall_sum	8	8	100

Table 9
Descriptive statistics for the understanding of the execution order dependencies.

Treatment/factor	Dependent variable	N	Max score	Mean	Median
Guided	Template1_ABSTRus	127	100	74.60	80.00
	Template1_EQUALus	127	100	71.92	77.80
	Template1_DETAILus	127	100	74.28	73.30
Unguided	Template1_ABSTRus	127	100	18.14	10.00
	Template1_EQUALus	127	100	12.50	0.00
	Template1_DETAILus	127	100	18.74	6.70

Table 10
Descriptive statistics for the understanding of the integration dependencies.

Treatment/factor	Dependent variable	N	Max score	Mean	Median
Guided	Template2_ABSTRus	127	100	64.90	100.00
	Template2_EQUALus	127	100	64.57	100.00
	Template2_DETAILus	127	100	74.80	100.00
Unguided	Template2_ABSTRus	127	100	21.00	0.00
	Template2_EQUALus	127	100	47.24	0.00
	Template2_DETAILus	127	100	49.61	0.00

Table 11
Descriptive statistics for the comprehension and recall tests.

Treatment	Control variable	N	Max score	Mean
Guided	Comprehension_sum	127	100	65.07
	Recall_sum	127	100	49.41
Unguided	Comprehension_sum	127	100	51.74
	Recall_sum	127	100	46.85

Table 11 shows the descriptive statistics for the comprehension (Comprehension_sum) and recall tasks (Recall_sum). The mean score for the Comprehension_sum is 65.07 for the guided treatment, and 51.74 for the unguided treatment. Next, the mean score for the Recall_sum is 49.41 for the guided treatment, and 46.85 for the unguided treatment.

Table 12 shows descriptive statistics for measures for the perceived ease of use and usefulness.

We used the Kolmogorov–Smirnov test to investigate the normality of our data. The Kolmogorov–Smirnov test shows that the data for all variables listed in Table 8 are non-parametric. The distribution scores for all dependent variables are significantly different from a normal distribution. Therefore, we use non-parametric test in the continuation.

6.2. Testing hypotheses on execution order dependencies

The Mann–Whitney test is a non-parametric equivalent to the independent t-test. Table 13 summarizes the data after the scores for Template1_ABSTRus, Template1_EQUALus, and Template1_DETAILus were ranked. The mean rank for Template1_ABSTRus exposed to the guided treatment is 184.70 and for the unguided treatment it is 70.30. The mean rank for Template1_EQUALus exposed to the guided treatment is 185.67 and for the unguided treatment it is 69.33. The mean rank for Template1_DETAILus exposed to the guided treatment is 180.44 and for the unguided treatment it is 74.56.

Table 13
Mann–Whitney test ranks for execution order dependency understanding.

	Treatment	N	Mean rank	Sum of ranks
Template1_ABSTRus	Guided	127	184.70	23456.50
	Unguided	127	70.30	8928.50
	Total	254		
Template1_EQUALus	Guided	127	185.67	23580.00
	Unguided	127	69.33	8805.00
	Total	254		
Template1_DETAILus	Guided	127	180.44	22915.50
	Unguided	127	74.56	9469.50
	Total	254		

Table 12
Descriptive statistics for measures of ease of usefulness.

Informative variable	Guided treatment Mean	Std. dev.	Unguided treatment Mean	Std. dev.
EaseOfUnderstand (1–7)	4.13	1.67	2.5	1.51
Frustrating (1–7)	3.76	1.67	4.91	1.91
EaseOfUse (1–7)	3.98	1.54	2.31	1.20
EaseOfLearning (1–7)	4.28	1.59	2.49	1.33

Table 14
Mann–Whitney test statistics for execution order dependency understanding.

	Template1_ABSTRus	Template1_EQUALus	Template1_DETAILus
Mann–Whitney U	800.50	677.00	1341.50
Wilcoxon W	8928.50	8805.00	9469.50
Z	–12.44	–12.78	–11.61
R	–0.78	–0.80	–0.73
Asymp.Sig (2-tailed)	0.000	0.000	0.000
Exact Sig (2-tailed)	0.000	0.000	0.000
Exact Sig (1-tailed)	0.000	0.000	0.000
Point Probability	0.000	0.000	0.000

Table 14 shows the Mann–Whitney test statistics for the three questions generated with Template 1. The grouping variable is TreatmentCode. Scores for Template1_ABSTRus exposed to the guided treatment (Mdn=80.00) were significantly higher than those exposed to the unguided treatment (Mdn=10.00) with $U=800.50$, $z=-12.44$, $p<0.05$, $r=-0.78$. Since the absolute value of r is greater than 0.5, the effect accounts for more than 25% of the variance, which is generally treated as a large effect size. This results support hypothesis H1.

Scores for Template1_EQUALus exposed to the guided treatment (Mdn=77.8) were significantly higher than those exposed to the unguided treatment (Mdn=0.00) with $U=677.00$, $z=-12.78$, $p<0.05$, $r=-0.80$. Since the absolute value of r is greater than 0.5, the effect accounts for more than 25% of the variance, which is generally treated as a large effect size. This results support hypothesis H2.

Scores for Template1_DETAILus exposed to the guided treatment (Mdn=73.3) were significantly higher than those exposed to the unguided treatment (Mdn=6.70) with $U=1341.50$, $z=-11.61$, $p<0.05$, $r=-0.73$. Since the absolute value of r is greater than 0.5, the effect accounts for more than 25% of the variance, which is generally treated as a large effect size. This results support hypothesis H3.

6.3. Testing hypotheses on integration dependencies

Table 15 summarizes the data after the scores for Template2_ABSTRus, Template2_EQUALus, and Template2_DETAILus were ranked. The mean rank for Template2_ABSTRus exposed to the guided treatment is 163.93 and for the unguided treatment it is

Table 15
Mann–Whitney test ranks for integration dependency understanding.

	Treatment	N	Mean rank	Sum of ranks
Template2_ABSTRus	Guided	127	163.93	20819.00
	Unguided	127	91.07	11566.00
	Total	254		
Template2_EQUALus	Guided	127	138.50	17589.00
	Unguided	127	116.50	14795.50
	Total	254		
Template2_DETAILus	Guided	127	143.50	18224.50
	Unguided	127	111.50	14160.50
	Total	254		

91.07. The mean rank for Template2_EQUALus exposed to the guided treatment is 138.50 and for the unguided treatment it is 116.50. The mean rank for Template2_DETAILus exposed to the guided treatment is 143.50 and for the unguided treatment it is 111.50.

Table 16 shows the Mann–Whitney test statistics for the three questions generated with Template 2. The grouping variable is TreatmentCode. Scores for Template2_ABSTRus exposed to the guided treatment (Mdn=100.00) were significantly higher than those exposed to the unguided treatment (Mdn=0.00) with $U=3438.00$, $z=-8.28$, $p<0.05$, $r=-0.52$. Since the absolute value of r is greater than 0.5, the effect accounts for more than 25% of the variance, which is generally treated as a large effect size. These results support hypothesis H4.

Scores for Template2_EQUALus exposed to the guided treatment (Mdn=100.00) were significantly higher than those exposed to the unguided treatment (Mdn=0.00) with $U=6667.00$, $z=-2.77$, $p<0.05$, $r=-0.17$. Since the absolute value of r is greater than 0.1, the effect accounts for between 1 and 9% of the variance, which is generally treated as a small effect size. These results support hypothesis H5.

Scores for Template2_DETAILus exposed to the guided treatment (Mdn=100.00) were significantly higher than those exposed to the unguided treatment (Mdn=0.00) with $U=6032.50$, $z=-4.13$, $p<0.05$, $r=-0.26$. Since the absolute value of r is greater than 0.1, the effect accounts for between 1 and 9% of the variance, which is generally treated as a small effect size. These results support hypothesis H6.

6.4. Validity

6.4.1. External validity

External validity threats are conditions that limit the ability to generalize the results of the experiment to industrial practice [74]. It is analyzed according to three aspects. The first aspect is concerned with how generalizable the results are to the wider population [74]. According to Gemino and Wand [68], two dimensions reflect several types of subjects in the conceptual modeling phase of scope definition: prior domain knowledge and prior modeling knowledge. We perceive our subjects as experts in modeling knowledge and as novices in prior domain knowledge. Similar to Batra et al. [72], we generalize the subjects to the population of typical information systems users who are involved in the analysis and design process, and

Table 16
Mann–Whitney test statistics for integration dependency understanding.

	Template2_ABSTRus	Template2_EQUALus	Template2_DETAILus
Mann–Whitney U	3438.00	6667.50	6032.50
Wilcoxon W	11566.00	14795.50	14160.50
R	–0.52	–0.17	–0.26
Z	–8.28	–2.77	–4.13
Asymp.Sig (2-tailed)	0.000	0.006	0.000
Exact Sig (2-tailed)	0.000	0.008	0.000
Exact Sig (1-tailed)	0.000	0.004	0.000
Point Probability	0.000	0.002	0.000

need to communicate about the execution order and integration dependencies among previously defined user stories.

The second aspect of the external validity of the presented research relates to the extent to which the material used is representative of real-world material [74]. The business process models used for Case 1 and Case 2 are part of a real business process model repository of a bank, which increases the external validity. The business process models were made for the purpose of business analysis and not for development reasons. This makes them perfect for studying the possibilities of reuse of existing documentation. On the other hand, our user story models are fictional. Their creation was inspired by the chosen business process models which reduces the external validity.

Finally, the third important aspect of the external validity relates to the time of carrying out the experiment [74]. All seven repetitions of the experiment were conducted in the same week. Nevertheless, some classes were scheduled in the mornings (starting at 8:00 at the earliest) and some in the afternoons (starting at 17:00 at the latest) which could have influenced how tired the students felt when participating in the experiment. We did not collect any information about their feelings at the moment the experiment was conducted.

6.4.2. Internal validity

Internal validity demands that the treatment causes the effect [74]. We considered two aspects. The first aspect of internal validity relates to maturation which has the effect that the subject reacts differently as time passes [74]. We primed them by giving them verbal explanations during the guided treatment. On the other hand, they were affected negatively when taking the unguided treatment since it was always scheduled as the second one and it did not contain any verbal explanations. Therefore, the students could have felt tired by the time they engaged in the unguided treatment. Nevertheless, the maximum time for each treatment was set at 24 min and all subjects in the class started both the first and second case together. They were encouraged to work through to the end of the workbook and were additionally motivated with bonus points for good performances in both treatments.

The second aspect of internal validity relates to resentful demoralization [74]. This means that a student exposed to less desirable treatment may give up on finishing the workbooks or not perform as well as they could have. The number of students who provided no entries for the unguided treatment is 0. Further, the number of subjects who provided no entries for any the six problem-solving questions of the

unguided treatment is 10. In order to investigate how much effect on the overall results those 10 subjects would have had we calculated all statistics with them excluded. The results showed that the data with 117 subjects support hypotheses H1, H2, H3, and H4 with a large effect size just like the data with 127 subjects do. Similarly, the data with 117 subjects support hypotheses H5 and H6 with a small effect size just like the data with 127 subjects do. We thus conclude that the bias in the perception of the subjects is insignificant.

6.4.3. Conclusion validity

Conclusion validity is concerned with the relationship between the treatment and the results, and the conclusions drawn from the results [74]. We discuss three aspects. The first aspect concerns the appropriateness of the statistical tests. A Kolmogorov–Smirnov test confirmed that the normality assumption did not hold for any of the measures. Therefore, we used a non-parametric Mann–Whitney test [75].

The second aspect concerns the reliability of the experimental replications [74]. As mentioned, the experiment was conducted in seven classes which were managed by one of six different lecturers. Table 17 shows descriptive statistics for the seven repetitions of the experiment. In addition, we report the results of Levene's test of homogeneity of variance in Table 18. Therefore, we summed up all the scores of the variables listed in Table 8 for the guided and unguided treatments of each workbook. We grouped the scores of the subjects' guided and unguided treatments by the seven classes. For the variable Guided_sum, the variances are equal for all seven classes, namely C1, C2, C3a, C3b, C4, C5, C6, $F(6,120)=1.62$; $p>0.05$. For the variable Unguided_sum, the variances for all seven classes are also equal, $F(6,120)=1.43$; $p>0.05$. We conclude that there are no significant statistical differences between the classes. Therefore, the replications are reliable.

The third aspect concerns the reliability of the experimental measures [74]. The idea is that measures which can be repeated with the same results are more reliable. Our workbooks consisted of two different cases managed by two different treatments. Table 19 shows descriptive statistics for Case 1 and Case 2. For Case 1, the average score for the guided treatment is higher (Guided_sum=528.25) than for the unguided treatment (Unguided_sum=224.76). For Case 2, the average score for the guided treatment is higher (Guided_sum=550.63) than for the unguided treatment (Unguided_sum=307.53). We conducted Levene's test of homogeneity of variances for Case 1 and Case 2, see

Table 17
Descriptive statistics for the experiment's seven repetitions.

Class' code	No. of subj.	Max score	Guided treatment (Guided_sum) Mean	Std. Dev.	Unguided treatment (Unguided_sum) Mean	Std. Dev.
C1	25	800	556.30	151.02	247.50	134.63
C2	11	800	593.75	169.59	215.64	141.72
C3a	21	800	488.43	183.56	206.34	137.83
C3b	20	800	526.01	138.67	282.08	145.60
C4	19	800	492.07	163.10	298.78	114.72
C5	18	800	622.54	123.19	340.87	153.30
C6	13	800	519.13	154.77	262.48	92.10

Table 18
Test of the homogeneity of variances for the seven repetitions of the experiment.

		Levene statistic	df1	df2	Sig.
Guided_sum	Based on Mean	1.623	6	120	0.147
	Based on Median	1.075	6	120	0.381
	Based on Median and with adjusted df	1.075	6	105.024	0.382
	Based on trimmed mean	1.565	6	120	0.163
Unguided_sum	Based on Mean	1.435	6	120	0.207
	Based on Median	0.936	6	120	0.472
	Based on Median and with adjusted df	0.936	6	102.301	0.473
	Based on trimmed mean	1.374	6	120	0.231

Table 19

Descriptive statistics for the different cases the subjects took.

Case	Max score	Guided treatment (Guided_sum) Mean	Std. dev.	N	Unguided treatment (Unguided_sum) Mean	Std. dev.	N
Case 1	800	528.25	171.31	63	550.63	145.48	64
Case 2	800	224.76	110.57	64	307.53	150.64	63

Table 20

Test of the homogeneity of variances for Case 1 and Case 2.

		Levene Statistic	df1	df2	Sig.
Guided_sum	Based on Mean	1.902	1	125	0.170
	Based on Median	1.903	1	125	0.298
	Based on Median and with adjusted df	1.903	1	111.980	0.298
	Based on trimmed mean	1.731	1	125	0.191
Unguided_sum	Based on Mean	10.521	1	125	0.002
	Based on Median	10.278	1	125	0.002
	Based on Median and with adjusted df	10.278	1	120.576	0.002
	Based on trimmed mean	10.526	1	125	0.002

Table 20. For the Guided_sum, the variances were equal for Case 1 and Case 2, $F(1,125)=1.9$; $p>0.05$. Therefore, the homogeneity of variances suggests that both Case 1 and Case 2 gave similar results when the subjects were exposed to the guided treatment. On the other hand, for the unguided treatment we cannot draw a similar conclusion. For Unguided_sum, the variances were significantly different for Case 1 and Case 2, $F(1,125)=10.52$; $p<0.01$, which increases the threat to the reliability of the measures and, therefore, represents a limitation on our research.

6.4.4. Construct validity

Construct validity threats relate to the design of the experiment and to the social factors [74]. We discuss three aspects. The first aspect is about the risk of a measurement bias [74]. In our experiment we used different types of measures (e.g. comprehension instruments, recall instruments, background questions) that were cross-checked against each other. A second aspect relates to a threat where the subject is involved in more than one study since treatments from the different studies may interact [74]. In our study, the subjects were involved in both guided and unguided treatments but with different cases. A third aspect is related to a threat called experimenter expectancies [74]. Experimenters can bias the results of a study both consciously and unconsciously based on what they expect from the experiment. The scoring of our workbooks was done by one of the authors, which increases the threat. Nevertheless, the scoring was based on a previously prepared list of master answers for both Case 1 and Case 2. The answers were set to be short and straightforward.

7. Discussion

7.1. Summary of the results

Our experimental study provides support for all six hypotheses – see the summary of results in Table 21. Hypotheses H1, H2, and H3 are focused on understanding the execution order dependencies among user stories. We demonstrated that the use of associated business process models together with the list of user stories (the outcome of the BuPUS) increases the understanding of the execution order dependencies and that the effect size is large. The results are in line with arguments built on the grounds of multimedia learning and with the comparison of grammars. We demonstrated that the sequence flow element (an arrow) used in associated business process models offers valuable information about the execution order dependencies among user stories.

Hypotheses H4, H5, and H6 are focused on understanding the integration dependencies among user stories. The understanding hap-

Table 21

Summary of the results.

Alternative hypothesis	Result	Effect size r
H1	Supported	–0.78 (large effect)
H2	Supported	–0.80 (large effect)
H3	Supported	–0.73 (large effect)
H4	Supported	–0.52 (large effect)
H5	Supported	–0.17 (small effect)
H6	Supported	–0.26 (small effect)

pens in two steps: first, when the subject understands correctly if the list of user stories contains user stories which are related on different levels of granularity and, second, when the subject understands correctly how those user stories are related. The statistics supported the results with different effect sizes. For H4 there is a large effect size, while for H5 and H6 there is a small one. The results are in line with the arguments.

7.2. Implications

The findings hold important *implications for research*. First, agile practices on requirements dependencies are a rather unexplored topic in the literature and empirical studies on it are scarce [10]. We have contributed an empirical study on requirement dependencies. In this way, we provide further evidence of the importance of execution order and integration dependencies for the understanding of user stories. Our findings underline the need to augment user stories with a business process perspective in order to provide the required context. This is an important finding for agile development research community.

Second, our research also complements research into project-specific software development methods. BuPUS demonstrates the feasibility of respective proposals in the specific agile development context. In this way, our empirical evidence lends further support for the specification method proposed by Bajec et al. [57].

Third, our research also contributes to the ongoing research stream on empirical software engineering. Several experiments have already been conducted in this area. We adopt task types to measure problem-solving, which were originally defined by Mayer to study his theory of multimedia learning [32]. These types of tasks are specifically useful in this context since they are highly relevant in practice for software requirements (“what can go wrong”).

Our research has direct *implications for the practice* of agile development. We addressed an important problem which arises from managing user stories of projects; namely, the missing context of a

user story. One way to investigate the context is by understanding dependencies of a user story. Our proposed method reuses existing business process models to provide information about the dependencies. Reusing existing documentations is one of the agile principles [3]. Therefore, it is suggested to augment agile development projects that use user stories with business process models for explicitly defining the user stories' context.

8. Conclusion

In this paper, we have proposed a method for better understanding the execution order and integration dependencies among user stories. The BuPUS builds upon existing business process models and associates user stories with corresponding business process activity elements. Using an experimental research design, we found that the understanding of the execution order and integration dependencies among user stories is greater when using BuPUS with partially large effect sizes. These findings are important to further advance agile development in research and practice.

More research is needed which would change the assumptions of our research. The assumption of our paper is that two independent documents exist: the list of user stories and the (corresponding) business process model, which are then integrated using BuPUS. First research question that we propose as further work is whether it would be possible to use business process models to generate a sufficiently good list of user stories. Recent research has developed methods to automatically generate textual descriptions from business process models [76]. User stories of Level 2 could be automatically generated from the activity elements. We believe they can serve as good candidates to brainstorm user stories of Level 1 and Level 3. Our second research question is how to use enterprise architecture models (especially models from the application and technology layer) to better understand different kind of dependencies among user stories. In that context, it would be interesting to conduct empirical research on dependencies beyond the process-related ones as in this paper, for example by including architectural dependencies [40]. This questions call for further experiments and an empirical research agenda.

Acknowledgments

We thank our colleagues Monika Malinova, Nikolai Neumayer, Andreas Rogge-Solti, Selim Erol, Reinhard Fisher, and Johann-Rudolf Göpfrih who all teach the Business Information Systems II course at Vienna University of Economics and Business in Austria.

References

- [1] H. Lindström, M. Malmsten, User-centred design and agile development: rebuilding the Swedish national union catalogue, in: J. McIntosh (Ed.), *Cataloging and Indexing: Challenges and Solutions*, CRC Press, 2011, pp. 252–266. ISBN: 1466562064, 9781466562066.
- [2] T. Dybå, T. Dingsoyr, Empirical studies of agile software development: a systematic review, *Inf. Softw. Technol.* 50 (9–10) (2008) 833–859, doi:10.1016/j.infsof.2008.01.006.
- [3] S. Ambler, *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*, Wiley Computer Publishing, New York, 2002, p. 402. ISBN: 0-471-20282-7.
- [4] P. Heck, A. Zaidman, A quality framework for agile requirements: a practitioner's perspective (2014) 11 <http://arxiv.org/pdf/1406.4692.pdf>.
- [5] D. Leffingwell, *Agile software requirements: lean requirements practices for teams, programs, and the enterprise*, in: A. Cockburn, J. Highsmith (Eds.), *Agile Software Development Series*, Addison-Wesley, Boston, USA, 2011 ISBN: 9780321635846.
- [6] M. Cohn, User stories applied for agile software development, in: K. Beck, M. Cohn, M. Flower (Eds.), *The Addison-Wesley Signature Series*, Addison-Wesley, Boston, USA, 2004, p. 268. ISBN: 0321205685.
- [7] W.C. Wake, *Extreme programming explored. XP series*, Addison-Wesley, 2002, p. 159. ISBN: 0201733978, 9780201733976.
- [8] Cohn, M., *Effective user stories, extreme programming and agile methods - XP/agile universe 2004*, ed. Zannier, C., H. Erdogmus, and L. Lindstrom, Springer, Calgary, Canada, 2004, pp. 208–208, ISBN: 3-540-22839-X.
- [9] J. Lin, H. Yu, Z. Shen, C. Miao, Using goal net to model user stories in agile software development, in: *Proceedings of IEEE/ACIS international conference on software engineering, artificial intelligence, networking and parallel/distributed computing*, Las Vegas, NV, IEEE, 2014, pp. 1–6, doi:10.1109/snpd.2014.6888731.
- [10] A. Martakis, M. Daneva, Handling requirements dependencies in agile projects: a focus group with agile software development practitioners, in: *Proceedings of International conference on research challenges in information science*, Paris, IEEE, 2013, pp. 1–11, doi:10.1109/RCIS.2013.6577679.
- [11] V. Kulshreshtha, J. Boardman, D. Verma, The emergence of requirements networks: the case for requirements inter-dependencies, *Int. J. Comput. Appl. Technol.* 45 (1) (2012) 28–41, doi:10.1504/IJCAT.2012.050130.
- [12] J. Li, Z. Liming, J. Ross, Y. Liu, H. Zhang, Q. Wang, M. Li, An initial evaluation of requirements dependencies types in change propagation analysis, *Proceedings of the International Conference On Evaluation & Assessment In Software Engineering*, IET, Ciudad Real, 2012, pp. 62–71, doi:10.1049/ic.2012.0009.
- [13] Y. Yan, S. Li, W. Sun, Dependency based technique for identifying the ripple effect on requirements evolution, *J. Softw.* 7 (3) (2012) 544–550, doi:10.4304/jsw.7.3.544-550.
- [14] C. Gupta, Y. Singh, D. Chauhan, *Dependency based process model for impact analysis: a requirement engineering perspective*, *Int. J. Comput. Appl.* 6 (6) (2010) 28–33.
- [15] A. Harrmann, M. Daneva, Requirements prioritization based on benefit and cost prediction: an agenda for future research, *International requirements engineering*, IEEE, Catalunya (2008) 125–134, doi:10.1109/RE.2008.48.
- [16] Y. Zgang, M. Marman, S.L. Lim, Empirical evaluation of search based requirements interaction management, *Inf. Softw. Technol.* 55 (1) (2013) 126–152, doi:10.1016/j.infsof.2012.03.007.
- [17] A. Cockburn, *Agile software development*, in: A. Cockburn, J. Highsmith (Eds.), *The Agile Software Development Series*, Pearson education, 2000.
- [18] R.J. Clarke, K. Kautz, What's in a user story: IS development methods as communication, *Information systems development: transforming organisations and society through information systems*, in: V. Strahonja, N. Vrček, D. Plantak Vukovac, C. Barry, M. Lang, H. Linger, C. Schneider (Eds.), *Faculty of Organization and Informatics, Varaždin, Croatia, 2014* ISBN: 978-953-6071-43-2.
- [19] O. Liskin, R. Pham, S. Kiesling, K. Schneider, Why we Need a Granularity Concept for User Stories, *Agile Processes in Software Engineering and Extreme Programming*, Springer International Publishing, 2014, pp. 110–125. ISBN: 978-3-319-06861-9.
- [20] A. Milicic, S. El Kadiri, A. Perdikakis, P. Ivanov, D. Kiritis, Toward the definition of domain concepts and knowledge through the application of the user story mapping method, *Int. J. Prod. Lifecycle Manag.* 7 (1) (2014) 3–16.
- [21] J. Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*, in: P. Economy (Ed.), O'Reilly, 2014 ISBN: 978-1-491-90490-9.
- [22] A. Saghaei, Y. Wand, Do ontological guidelines improve understandability of conceptual models? A meta-analysis of empirical work, in: *Proceedings of the Hawaii International Conference on System Sciences*, Waikoloa, HI, IEEE, 2014, doi:10.1109/HICSS.2014.567.
- [23] D. Lubke, K. Schneider, M. Weidlich, Visualizing use case sets as BPMN processes, *Requirements Engineering Visualization*, IEEEBarcelona, Catalunya, 2008, pp. 21–25, doi:10.1109/REV.2008.8.
- [24] Mendling, J., Strembeck, M., and Recker, J., Factors of process model comprehension - findings from a series of experiments, *Decis. Support Syst.*, 53 (1), 2012, pp. 195–206, DOI: 10.1016/j.dss.2011.12.013.
- [25] A. Ottensooser, A. Fekete, H.A. Reijers, J. Mendling, C. Menictas, Making sense of business process descriptions: an experimental comparison of graphical and textual notations, *J. Syst. Softw.* 85 (3) (2012) 596–606, doi:10.1016/j.jss.2011.09.023.
- [26] OMG, *Business Process Model and Notation (BPMN), v2.0*, (2011) 550 <http://www.omg.org/spec/BPMN/2.0>.
- [27] J. Recker, Empirical investigation of the usefulness of Gateway constructs in process models, *Eur. J. Inf. Syst.* 22 (2013) 673–689, doi:10.1057/ejis.2012.50.
- [28] H.A. Reijers, J. Mendling, A study into the factors that influence the understandability of business process models, *IEEE Trans. Syst. Man and Cybern.* 41 (3) (2011) 449–462, doi:10.1109/tsmca.2010.2087017.
- [29] P. Bera, A. Burton-Jones, Y. Wand, Research note - How semantics and pragmatics interacts in understanding conceptual models, *Inf. Syst. Res.* 25 (2) (2014) 401–419, doi:10.1287/isre.2014.0515.
- [30] J. Mendling, H.A. Reijers, J. Recker, Activity labeling in process modeling: empirical insights and recommendations, *Inf. Syst.* 35 (2010) 467–482, doi:10.1016/j.is.2009.03.009.
- [31] J. Mendling, *Metrics for process models: empirical foundations of verification, error prediction, and guidelines for correctness*, *Lecture Notes in Business Information Processing*, Springer, Berlin Heidelberg, 2008 ISBN: 978-3-540-89223-6.
- [32] R.E. Mayer, *Multi-Media Learning*, Second edition, Cambridge university press, USA, 2009 ISBN: 9780521735353.
- [33] P. Wagstrom, J. Herbsleb, Dependency forecasting in the distributed agile organization, *Commun. ACM* 49 (10) (2006) 55–56, doi:10.1145/1164394.1164420.
- [34] D.E. Strode, A dependency taxonomy for agile software development projects, *Inf. Syst. Front.* * (*) (2015) 1–24, doi:10.1007/s10796-015-9574-1.
- [35] M. Daneva, E. van der Veen, C. Amrit, S. Ghaisas, K. Sikkel, R. Kumar, N. Ajmeri, U. Ramteerthkar, R. Wieringa, Agile requirements prioritization in large-scale outsourced system projects: an empirical study, *J. Syst. Softw.* 86 (2013) 1333–1353, doi:10.1016/j.jss.2012.12.046.
- [36] M. Cataldo, A. Mockus, J.A. Roberts, J.D. Herbsleb, Software dependencies, work dependencies, and their impact on failures, *IEEE Trans. Soft. Eng.* 35 (6) (2009) 864–878, doi:10.1109/TSE.2009.42.

- [37] S. Coyle, K. Canboy, A case study of risk management in agile systems development, in: *Proceedings of the European Conference on Information Systems NUI Galway, Verona, Italy, 2009*, pp. 2567–2578.
- [38] H.F. Cervone, Project risk management, *OCLC Syst. Serv.: Int. Digit. Libr. Perspect.* 22 (4) (2006) 256–262, doi:10.1108/10650750610706970.
- [39] Z. Racheva, M. Daneva, K. Sikkil, R. Wieringa, Do we know enough about requirements prioritization in agile projects: insights from a case study, in: *Proceedings of International Requirements Engineering Conference, IEEE, Sydney, 2010*, pp. 147–156.
- [40] A. Gomez, G. Rueda, P.P. Alarcón, A systematic and lightweight method to identify dependencies between user stories, agile processes in software engineering and extreme programming, *Business Information Processing*, Springer, Berlin, 2010, pp. 190–195. Lecture notes ISBN: 978-3-642-13053-3.
- [41] K. Beck, Embracing change with extreme programming, *Computer* 32 (10) (1999) 70–77, doi:10.1109/2.796139.
- [42] B. Deckter, E. Ras, J. Rech, P. Jaubert, M. Rieth, Wiki-based stakeholder participation in requirements engineering, *IEEE Softw.* 24 (2) (2007) 28–35, doi:10.1109/MS.2007.60.
- [43] M. Paasivaara, V.T. Heikkilä, C. Lassenius, Experiences in scaling the product owner role in large-scale globally distributed Scrum, in: *Proceedings of the International Conference on Global Software Engineering, Porto Alegre, IEEE, 2012*, pp. 174–178, doi:10.1109/ICGSE.2012.41.
- [44] V. Mahnič, T. Hovelja, On using planning poker for estimating user stories, *J. Syst. Softw.* 85 (9) (2012) 2086–2095, doi:10.1016/j.jss.2012.04.005.
- [45] M. Dumas, M. La Rosa, J. Mendling, H. Reijers, *Fundamentals of Business Process Management*, Springer-Verlag, Berlin Heidelberg, Berlin, Germany, 2013, p. 399. ISBN: 978-3-642-33142-8.
- [46] P. Trkman, The critical success factors of business process management, *Int. J. Inf. Manag.* 30 (2) (2010) 125–134, doi:10.1016/j.jinfomgt.2009.07.003.
- [47] I. Davies, P. Green, M. Rosemann, M. Indulska, S. Gallo, How do practitioners use conceptual modeling in practice? *Data & Knowl. Eng.* 58 (3) (2006) 358–380, doi:10.1016/j.datak.2005.07.007.
- [48] C.M. DaSilva, P. Trkman, Business model: what it is and what it is not, *Long Range Plan.* 47 (6) (2014) 379–389, doi:10.1016/j.lrp.2013.08.004.
- [49] A. Pivk, O. Vasilecas, D. Kalibatiene, R. Rupnik, On approach for the implementation of data mining to business process optimisation in commercial companies, *Technol. Econ. Dev. Econ.* 19 (2) (2013) 237–256, doi:10.3846/20294913.2013.796501.
- [50] A. Dreiling, M. Rosemann, W.M.P. van der Aalst, S. Sadiq, From conceptual process models to running systems: a holistic approach for the configuration of enterprise system processes, *Decis. Support Syst.* 45 (2) (2008) 189–207, doi:10.1016/j.dss.2007.02.007.
- [51] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and design*, Prentice Hall Professional Reference, Upper Saddle River, New Jersey, USA, 2005 ISBN: 0-13-185858-0.
- [52] M. Jurič, K. Pant, *Business process driven SOA Using BPMN and BPEL: from Business Process Modeling to Orchestration and Service Oriented Architecture*, Packt Publishing, Birmingham, England, 2008, p. 328. ISBN: 9781847191465.
- [53] P. Trkman, A. Kovačič, A. Popovič, SOA adoption phases: a case study, *Bus. Inf. Syst. Eng.* 3 (4) (2011) 211–220, doi:10.1007/s12599-011-0168-2.
- [54] D. Liu, M. Shen, Business-to-business workflow interoperability based on process views, *Decis. Support Syst.* 38 (3) (2004) 399–419, doi:10.1016/S0167-9236(03)00116-7.
- [55] R.M. Dijkman, M. La Rosa, H.A. Reijers, Managing large collections of business process models – current techniques and challenges, *Comput. Ind.* 63 (2) (2012) 91–97, doi:10.1016/j.compind.2011.12.003.
- [56] A.S.M. Tam, L.K. Chu, D. Sculli, Business process modelling in small- to medium-sized enterprises, *Ind. Manag. Data Syst.* 101 (4) (2001) 144–152, doi:10.1108/02635570110390107.
- [57] M. Bajec, D. Vavpotič, M. Krisper, Practice-driven approach for creating project-specific software development methods, *Inf. Softw. Technol.* 49 (2007) 343–365, doi:10.1016/j.infsof.2006.05.007.
- [58] A.F. Hermesen, S. Brinkkemper, H. Oei, Situational method engineering for information system project approaches, IFIP WG 8.1 working conference, in: *Stuart, A.A. Verrijn, T.W. Olle (Eds.), IFIP Transactions A-55, Maastricht, Netherlands, 1994*, pp. 169–194. ISBN: 0-444-82074-4.
- [59] G. Shanks, E. Tansley, R. Weber, Using ontology to validate conceptual models, *Commun. ACM – Serv. Oriented Comput.* 46 (10) (2003) 85–89, doi:10.1145/944217.944244.
- [60] Wand, Y., Ontology as a foundation for meta-modelling and method engineering, *Inf. Softw. Technol.*, 38 (4), 1996, pp. 281–287, doi: 10.1016/0950-5849(95)01052-1.
- [61] D. Avison, G. Fitzgerald, *Information Systems Development – Methodologies, Techniques, and Tools*, 4th edition, McGraw-Hill Education, Berkshire, UK, 2006, p. 645. ISBN: 0077096266.
- [62] N. Lohmann, E. Verbeek, R. Dijkman, Petri net transformations for business processes – a survey, in: K. Jensen, W. van der Aalst (Eds.), *Transactions on Petri Nets and other Models of Concurrency II, Lecture notes in Computer Science*, Springer, Berlin Heidelberg, Berlin, Germany, 2009, pp. 46–63. ISBN: 978-3-642-00898-6.
- [63] P. Loucopoulos, E. Kavakli, Enterprise modelling and the teleological approach to requirements engineering, *Int. J. Cooperative Inf. Syst.* 4 (1) (1995) 45–79, doi:10.1142/S0218843095000032.
- [64] M. zur Muehlen, J. Recker, How much language is enough? Theoretical and practical use of the business process modeling notation, in: Z. Bellahsene, M. Léonard (Eds.), *Advance Information Systems Engineering, Lecture notes in Computer Science*, Springer, Berlin Heidelberg, Berlin, Germany, 2008, pp. 456–479. ISBN: 978-3-540-69533-2.
- [65] J.C. Recker, M. Indulska, M. Rosemann, P. Green, Do process modelling techniques get better? A comparative ontological analysis of BPMN, in: *Proceedings of the Australasian Conference on Information Systems, QUT ePrints, Sydney, Australia, 2013*.
- [66] J. Mendling, H.A. Reijers, W.M.P. van der Aalst, Seven process modeling guidelines, *Inf. Softw. Technol.* 52 (2) (2010) 127–137, doi:10.1016/j.infsof.2009.08.004.
- [67] H. Leopold, J. Mendling, O. Gunther, What we can learn from quality issues of BPMN models from industry, *IEEE Softw.* (2015) 1–9 PP (99), doi:10.1109/MS.2015.81.
- [68] A. Gemino, Y. Wand, A framework for empirical evaluation of conceptual modeling techniques, *Requir. Eng.* 9 (4) (2004) 248–260, doi:10.1007/s00766-004-0204-6.
- [69] Y. Wand, R. Weber, On the ontological expressiveness of information systems analysis and design grammars, *Inf. Syst. J.* 3 (4) (1993) 217–237, doi:10.1111/j.1365-2575.1993.tb00127.x.
- [70] J.H. Larkin, H.A. Simon, Why a diagram is (sometimes) worth ten thousand words, *Cognit. Sci.* 11 (1) (1987) 65–100, doi:10.1016/S0364-0213(87)80026-5.
- [71] R.J. Wieringa, *Design science Methodology for Information Systems and Software Engineering*, Springer, Heidelberg, 2014, p. 332. ISBN: 9783662438398.
- [72] D. Batra, J.A. Hoffer, R.P. Bostrom, Comparing representations with relational and EER models, *Commun. ACM* 33 (2) (1990) 126–139, doi:10.1145/75577.75579.
- [73] A. Maes, G. Poels, Evaluating quality of conceptual modelling scripts based on user perceptions, *Data & Knowl. Eng.* 63 (2007) 701–724, doi:10.1016/j.datak.2007.04.008.
- [74] C. Wohlin, P. Runeson, M. Host, M.C. Ohlsson, B. Regnell, A. Wesslen, *Experimentation in software engineering, an introduction*, in: V.R. Basili (Ed.), *The Kluwer International Series in Software Engineering*, Springer, Boston, 2000 ISBN: 978-3-642-29043-5.
- [75] A. Field, *Discovering Statistics using SPSS*, in: M. Carmichael (Ed.), *SAGE publications, Canada, 2009* ISBN: 987-1-4462-4917-8.
- [76] H. Leopold, J. Mendling, A. Polyvyanyy, Supporting process model validation through natural language generation, *IEEE Trans. Softw. Eng.* 40 (8) (2013) 818–840, doi:10.1109/TSE.2014.2327044.